14

Factoring polynomials over finite fields

In this chapter, we present several algorithms for the factorization of univariate polynomials over finite fields. The two central steps are distinct-degree factorization, where irreducible factors of distinct degrees are separated from each other, and equal-degree factorization, where all irreducible factors of the input polynomial have the same degree. The reader who is happy with the basic result of probabilistic polynomial-time factorization only has to go up to Section 14.4. The remaining sections discuss root finding (14.5), squarefree factorization (14.6), faster algorithms (14.7), methods using a different approach based on linear algebra (14.8), and the construction of irreducible polynomials and BCH codes (14.9 and 14.10). The implementations, briefly described in Section 15.7, show that this is an area where computer algebra has been tremendously successful: we can now factor enormously large polynomials.

14.1. Factorization of polynomials

The **fundamental theorem of number theory** states that every integer can be (essentially uniquely) factored as a product of primes. Similarly, for any field F the polynomials in $F[x_1,\ldots,x_n]$ can be (essentially uniquely) factored into a product of irreducible polynomials. In other words, $\mathbb{Z}$ and $F[x_1,\ldots,x_n]$ are **Unique Factorization Domains** (Sections 6.2, 25.2).

The "essentially uniquely" says that such a factorization is unique up to the order of the factors and multiplication by units, that is, by 1 or -1 in $\mathbb{Z}$ and by nonzero constants from F in $F[x_1,\ldots,x_n]$. As an example, $x^2-1=(x-1)(x+1)=(-x-1)(-x+1)$. A polynomial $f\in F[x_1,\ldots,x_n]$ is **irreducible** if and only if $f\notin F$ and for any $g,h\in F[x_1,\ldots,x_n]$ with $f=gh$ we have $g\in F$ or $h\in F$. We say that f is **squarefree** if it has no proper quadratic divisor, so that for any $g\in F[x_1,\ldots,x_n]$ with g^2 dividing f we have $g\in F$.

The problem of univariate polynomial factorization is, given a polynomial f in $F[x]$ for a field F, to determine pairwise distinct monic irreducible polynomials $f_1,\ldots,f_r\in F[x]$ and positive integers $e_1,\ldots,e_r\in\mathbb{N}$ such that $f=\mathrm{lc}(f)f_1^{e_1}\cdots f_r^{e_r}$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

It seems computationally difficult to factor large integers (Chapter 19). However, computer algebra systems can routinely factor reasonably large polynomials; this is a task where the usual computational analogies between integers and polynomials break down. The irreducible factors of a univariate polynomial of degree n have degree at most n, and over a finite field $\mathbb{F}_q$ with q elements (Section 25.4) there are only finitely many such polynomials. But an exhaustive search might consume roughly $q^{n/2}$ trials, and this is exponential both in n and the bit size $\log_2 q$ of the representation of a field element.

We will describe in detail probabilistic algorithms that factor a univariate polynomial of degree n over a finite field $\mathbb{F}_q$ in time polynomial in n and $\log q$, in fact, with about $n^2 + n \log q$ operations in $\mathbb{F}_q$. An interesting effect of having the two independent input size parameters n and $\log q$ is that today there are several asymptotically best algorithms, depending on the relation between the two parameters (Figure 14.9). More general questions include factoring polynomials in

- $\mathbb{Z}[x]$ and $\mathbb{Q}[x]$,
- $\mathbb{Q}(\alpha)[x]$, where $\mathbb{Q}(\alpha)$ is an algebraic number field (a finite algebraic extension of $\mathbb{Q}$),
- $\mathbb{Z}_m[x]$, where $m \in \mathbb{N}$ is a positive integer,
- $\mathbb{R}[x]$ and $\mathbb{C}[x]$,
- and multivariate polynomials.

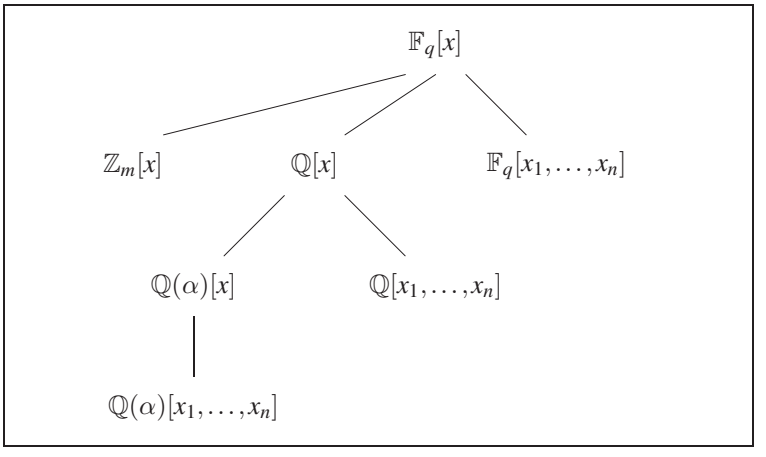


FIGURE 14.1: Polynomial factorization in various domains.

The dependencies between some of these are shown in Figure 14.1. It turns out that factoring univariate polynomials over finite fields is a basic task used in many other factoring algorithms. Factoring in $\mathbb{Q}[x]$ is the topic of Chapters 15 and 16.

Some algorithms for finite fields proceed in three stages:

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

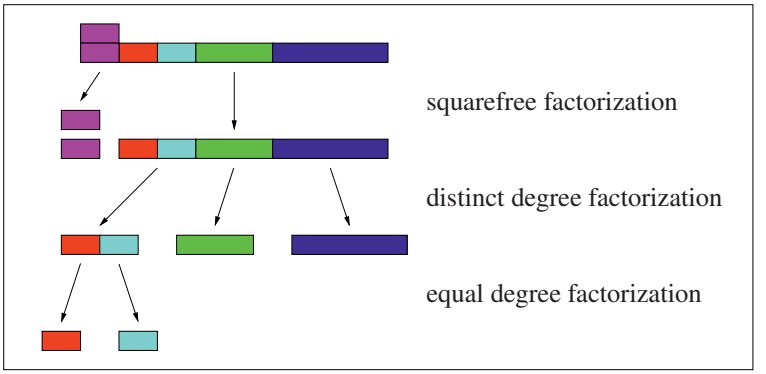


FIGURE 14.2: The stages of univariate polynomial factorization over finite fields.

1. squarefree factorization,
2. distinct-degree factorization,
3. equal-degree factorization.

Squarefree factorization gets rid of multiple factors, distinct-degree factorization splits irreducible factors according to their degrees, and equal-degree factorization solves the remaining problem, where all irreducible factors are distinct and of the same degree. In Figure 14.2, we see how the three stages work. The width of a box represents the degree of the corresponding polynomial; different colors stand for different irreducible factors. In the example, the original polynomial consists of four factors of degree 2 (two of them equal), one factor of degree 4, and one of degree 6.

The first stage is quite easy, both in theory and in practice. When the input is a large random polynomial, then the third stage is likely to be needed only for very small polynomials, and the second stage consumes the bulk of the computing time (more than 99% in our experiments described in Section 15.7).

In the next three sections we present in detail a conceptually simple complete factorization algorithm; see Figure 14.7. The determination of repeated factors in Figure 14.2 is actually delayed until the end.

A fundamental tool for our algorithms is the following theorem (see Section 25.4 for a proof), which generalizes Theorem 4.9.

THEOREM 14.1 *Fermat's little theorem.*
For nonzero $a \in \mathbb{F}_q$, we have $a^{q-1} = 1$, and for all $a \in \mathbb{F}_q$, we have $a^q = a$, and

$$x^q - x = \prod_{a \in \mathbb{F}_q} (x - a) \text{ in } \mathbb{F}_q[x].$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

The reader must be thoroughly familiar with the material of Section 25.4 on finite fields, of which we make substantial use. The notation $\mathbb{F}_q$ and Fermat's little theorem in $\mathbb{F}_q$ are used over and over again. We will also use the fact that if $f \in \mathbb{F}_q[x]$ is irreducible of degree n, then $\mathbb{F}_{q^n} = \mathbb{F}_q[x]/\langle f \rangle$ is a field with q^n elements (Section 4.2), and Fermat's little theorem implies that $\mathbb{F}_q = \{a \in \mathbb{F}_{q^n} : a^q = a\}$.

The possible sizes of finite fields are precisely the prime powers, and in the following, q always denotes a prime power. The reader may think of q as being a prime number. However, most statements or proofs do not become simpler for this special case, so that we may as well work in full generality.

14.2. Distinct-degree factorization

In this section, we describe the distinct-degree factorization stage for squarefree polynomials, deferring the question of how to deal with other polynomials until Section 14.4.

Fermat's little theorem is the special case $d = 1$ of the following result.

THEOREM 14.2.
For any $d \geq 1$, $x^{q^d} - x \in \mathbb{F}_q[x]$ is the product of all monic irreducible polynomials in $\mathbb{F}_q[x]$ whose degree divides d.

PROOF. By Fermat's little theorem 14.1, applied to $\mathbb{F}_{q^d}$, $h = x^{q^d} - x$ is the product of all $x - a$ with $a \in \mathbb{F}_{q^d}$. If g^2 divides h (over $\mathbb{F}_q$) with $g \in \mathbb{F}_q[x] \setminus \mathbb{F}_q$, then some $x - a$ divides g and $(x-a)^2$ divides h. Since this is impossible, no such g exists, and $x^{q^d} - x$ is squarefree. It is sufficient to show for any monic irreducible polynomial $f \in \mathbb{F}_q[x]$ of degree n that

$$f \text{ divides } x^{q^d} - x \iff n \text{ divides } d.$$

We consider the field extension $\mathbb{F}_q \subseteq \mathbb{F}_{q^d}$. If f divides $x^{q^d} - x$, then from Theorem 14.1, applied to $\mathbb{F}_{q^d}$, we get a set $A \subseteq \mathbb{F}_{q^d}$ with $f = \prod_{a \in A}(x - a)$. We choose some $a \in A$, and let $\mathbb{F}_q[x]/\langle f \rangle \cong \mathbb{F}_q(a) \subseteq \mathbb{F}_{q^d}$, where $\mathbb{F}_q(a)$ is the smallest subfield of $\mathbb{F}_{q^d}$ containing a (Section 25.3). This is a field with q^n elements, and $\mathbb{F}_{q^d}$ is an extension of $\mathbb{F}_q(a)$, so that $q^d = (q^n)^e$ for some integer $e \geq 1$. Hence n divides d.

Now suppose that n divides d, let $\mathbb{F}_{q^n} = \mathbb{F}_q[x]/\langle f \rangle$, and $a = (x \bmod f) \in \mathbb{F}_{q^n}$ be a root of f. Theorem 14.1 says that $a^{q^n} = a$. Since $q^n - 1$ divides $q^d - 1 = (q^n - 1) \cdot e$ with $e = q^{d-n} + q^{d-2n} + \cdots + 1$, also $x^{q^n - 1} - 1$ divides

$$x^{q^d - 1} - 1 = (x^{q^n - 1} - 1)(x^{(q^n - 1)(e-1)} + \cdots + 1).$$

Multiplying by x, we find that $x^{q^n} - x$ divides $x^{q^d} - x$, and hence

$$(x - a) \mid (x^{q^n} - x) \mid (x^{q^d} - x),$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

so that $x-a$ divides $\gcd(f, x^{q^d}-x)$ in $\mathbb{F}_{q^n}[x]$. But the gcd of two polynomials with coefficients in $\mathbb{F}_q$ also has coefficients in $\mathbb{F}_q$ (Example 6.19), and since it is nonconstant and f is irreducible, $\gcd(f, x^{q^d}-x) = f$, or, equivalently, f divides $x^{q^d}-x$. □

The **distinct-degree decomposition** of a nonconstant polynomial $f \in \mathbb{F}_q[x]$ is the sequence $(g_1, \ldots, g_s)$ of polynomials, where g_i is the product of all monic irreducible polynomials in $\mathbb{F}_q[x]$ of degree i that divide f and $g_s \neq 1$ (but some g_i for $i < s$ may be 1). As an example, (x^2+x, x^4+x^3+x+2) is the distinct-degree decomposition of $f = x(x+1)(x^2+1)(x^2+x+2) \in \mathbb{F}_3[x]$; the two quadratic factors are irreducible since they have no zeroes in $\mathbb{F}_3$.

ALGORITHM 14.3 Distinct-degree factorization.
Input: A squarefree monic polynomial $f \in \mathbb{F}_q[x]$ of degree $n > 0$.
Output: The distinct-degree decomposition $(g_1, \ldots, g_s)$ of f.

1. $h_0 \longleftarrow x, \quad f_0 \longleftarrow f, \quad i \longleftarrow 0$
   repeat
2. $i \longleftarrow i+1$
   call the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $h_i = h_{i-1}^q \text{ rem } f$
3. $g_i \longleftarrow \gcd(h_i - x, f_{i-1}), \quad f_i \longleftarrow \dfrac{f_{i-1}}{g_i}$
4. **until** $f_i = 1$
5. $s \longleftarrow i$
   return $(g_1, \cdots, g_s)$

The distinct-degree decomposition is a sequence of polynomials, and **distinct-degree factorization** is the process of computing this sequence.

In our cost analyses, we use a multiplication time M as on the inside back cover.

THEOREM 14.4.
The distinct-degree factorization algorithm works correctly as specified. It takes $O(s\mathsf{M}(n)\log(nq))$ or $O^{\sim}(n^2 \log q)$ operations in $\mathbb{F}_q$, where s is the largest degree of an irreducible factor of f.

PROOF. Let $(G_1, \ldots, G_t)$ be the distinct-degree decomposition of f, with $G_t \neq 1$. For the correctness, it is sufficient to show by induction on $i \geq 0$ that

$$h_i \equiv x^{q^i} \bmod f, \quad f_i = G_{i+1} \cdots G_t, \quad g_i = G_i \text{ if } i \geq 1.$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

The first two claims are clear for $i = 0$. For $i \geq 1$, we have $h_i \equiv h_{i-1}^q \equiv (x^{q^{i-1}})^q = x^{q^i} \bmod f$, so that $h_i - x \equiv x^{q^i} - x \bmod f$ and

$$g_i = \gcd(h_i - x, f_{i-1}) = \gcd(x^{q^i} - x, f_{i-1}).$$

By Theorem 14.2, g_i is the product of all monic irreducible polynomials in $\mathbb{F}_q[x]$ of degree dividing i that divide $f_{i-1} = G_i \cdots G_n$, hence $g_i = G_i$. Furthermore, $f_i = G_i \cdots G_n / g_i = G_{i+1} \cdots G_n$. This finishes the inductive step and also shows that $s = t$.

The cost for computing h_i in step 2 is $O(\log q)$ multiplications modulo f in step 2, or $O(\mathsf{M}(n) \log q)$ operations in $\mathbb{F}_q$, by Corollary 11.11. Similarly, the cost for computing g_i and f_i is $O(\mathsf{M}(n) \log n)$ operations. □

Algorithm 14.3 may be stopped as soon as $\deg f_i < 2(i+1)$, since all irreducible factors of f_i have degree at least $i+1$, and hence f_i is irreducible in that case. This is called **early abort** and guarantees that the algorithm stops after $i = \max\{m_1/2, m_2\} \leq n/2$, where m_1 and m_2 are the degrees of the largest and the second largest irreducible factor of f, respectively. In step 2, h_i is actually only needed modulo f_{i-1}.

EXAMPLE 14.5. We let $q = 3$ and trace Algorithm 14.3 on the squarefree polynomial $f = x^8 + x^7 - x^6 + x^5 - x^3 - x^2 - x \in \mathbb{F}_3[x]$. Then

$$\begin{aligned}
h_1 &= h_0^3 \operatorname{rem} f = x^3 \operatorname{rem} f = x^3, \\
g_1 &= \gcd(h_1 - x, f_0) = \gcd(x^3 - x, f) = x, \\
f_1 &= \frac{f_0}{g_1} = \frac{f}{x} = x^7 + x^6 - x^5 + x^4 - x^2 - x - 1, \\
h_2 &= h_1^3 \operatorname{rem} f = x^9 \operatorname{rem} f = -x^7 + x^6 + x^5 + x^4 - x, \\
g_2 &= \gcd(h_2 - x, f_1) = \gcd(-x^7 + x^6 + x^5 + x^4 + x, f_1) = x^4 + x^3 + x - 1, \\
f_2 &= \frac{f_1}{g_2} = \frac{x^7 + x^6 - x^5 + x^4 - x^2 - x - 1}{x^4 + x^3 + x - 1} = x^3 - x + 1.
\end{aligned}$$

At this point, Algorithm 14.3 would perform one further iteration, but the early abort condition $\deg f_2 < 2(2+1) = 6$ says that this is not necessary since f_2 is already irreducible. Thus f has one linear factor, two distinct irreducible quadratic factors (which we do not know yet), and one irreducible cubic factor. The trace is illustrated in Figure 14.3. ◇

14.3. Equal-degree factorization: Cantor and Zassenhaus' algorithm

The remaining task is **equal-degree factorization**: to factor one of the polynomials that are produced by the previous distinct-degree factorization. Our algorithm works only for an *odd* prime power q; a method for characteristic 2 is described in Exercise 14.16.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

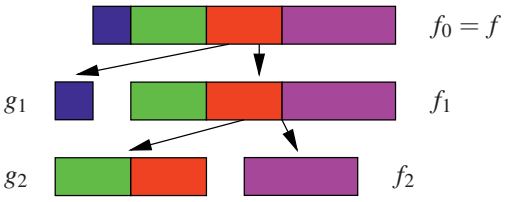


FIGURE 14.3: Sample distinct-degree factorization.

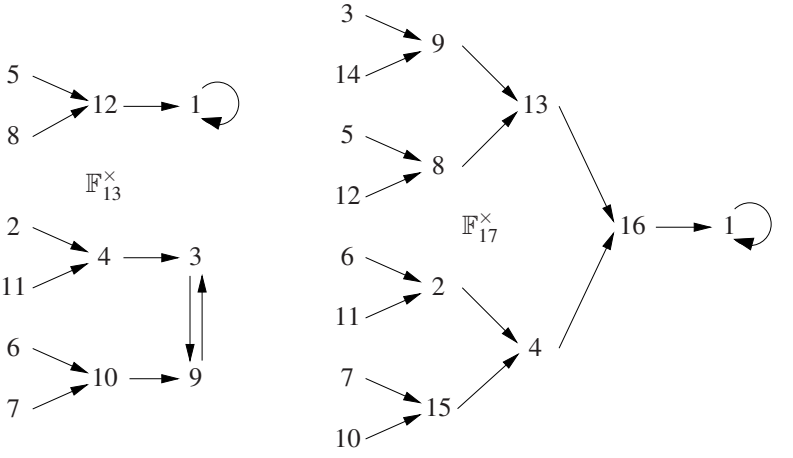


FIGURE 14.4: Squaring in $\mathbb{F}_{13}^{\times}$ and $\mathbb{F}_{17}^{\times}$.

We first collect some facts about the squaring map $\sigma: \mathbb{F}_q^{\times} \longrightarrow \mathbb{F}_q^{\times}$, with $\sigma(a) = a^2$. As an example, the effect of σ on the elements of $\mathbb{F}_{13}^{\times}$ and of $\mathbb{F}_{17}^{\times}$ is given in Figure 14.4. An arrow from a number i to a number j indicates that $j = \sigma(i)$. Each element has either two or zero arrows pointing to it; the first ones are the **squares**, the second ones the **nonsquares**. Both sets contain exactly half of the elements. Lemma 14.7 below, which is the special case $k = 2$ of the following lemma, says that this is always the case.

LEMMA 14.6. *Let q be a prime power, k a divisor of $q-1$, and $S = \{b^k : b \in \mathbb{F}_q^{\times}\}$ the set of kth powers in $\mathbb{F}_q^{\times}$.*

(i) S is a subgroup of order $(q-1)/k$.

(ii) $S = \{a \in \mathbb{F}_q^{\times} : a^{(q-1)/k} = 1\}$.

PROOF. S is the image of the kth power homomorphism $\sigma_k: \mathbb{F}_q^{\times} \longrightarrow \mathbb{F}_q^{\times}$ with $\sigma_k(b) = b^k$ and hence a subgroup of $\mathbb{F}_q^{\times}$. The kernel of σ_k is

$$\ker \sigma_k = \{a \in \mathbb{F}_q^{\times} : \sigma_k(a) = 1\} = \{a \in \mathbb{F}_q^{\times} : a^k = 1\}, \tag{1}$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

the set of kth roots of unity. Since $\mathbb{F}_q$ is a field, the polynomial $x^k - 1 \in \mathbb{F}_q[x]$ has at most k roots in $\mathbb{F}_q[x]$ (Lemma 25.4), and hence $\#\ker\sigma_k \le k$.

Since $(b^k)^{(q-1)/k} = b^{q-1} = 1$ for all $b \in \mathbb{F}_q^\times$, by Fermat's little theorem 14.1, we have $S \subseteq \ker\sigma_{(q-1)/k}$. By the same reasoning as above, this implies that $\#S \le (q-1)/k$. Now

$$q-1 = \#\mathbb{F}_q^\times = \#\ker\sigma_k \cdot \#\operatorname{im}\sigma_k = \#\ker\sigma_k \cdot \#S \le k\cdot(q-1)/k = q-1,$$

by the homomorphism theorem for groups, and this implies that $\#\ker\sigma_k = k$, $\#S = (q-1)/k$, and $S = \ker\sigma_{(q-1)/k}$. □

Applying the lemma for $k = 2$ and $k = (q-1)/2$, we obtain the following.

LEMMA 14.7. *Let q be an odd prime power and*

$$S = \{a \in \mathbb{F}_q^\times : \exists b \in \mathbb{F}_q^\times \ a = b^2\}$$

be the set of squares in $\mathbb{F}_q^\times$. Then

(i) $S \subseteq \mathbb{F}_q^\times$ is a (multiplicative) subgroup of order $(q-1)/2$,

(ii) $S = \{a \in \mathbb{F}_q^\times : a^{(q-1)/2} = 1\}$,

(iii) $a^{(q-1)/2} \in \{1,-1\}$ for all $a \in \mathbb{F}_q^\times$.

Now we want to factor a monic polynomial $f \in \mathbb{F}_q[x]$ with $\deg f = n$, and have a divisor $d \in \mathbb{N}$ of n so that each irreducible factor of f has degree d. There are $r = n/d$ such factors, and we can write $f = f_1 \cdots f_r$ with distinct monic irreducible $f_1, \ldots, f_r \in \mathbb{F}_q[x]$. We may assume that $r \ge 2$; otherwise, we know that f is irreducible. Since $\gcd(f_i, f_j) = 1$ for $i \ne j$, we have the ring isomorphism of the Chinese Remainder Theorem 5.3:

$$\chi : R = \mathbb{F}_q[x]/\langle f\rangle \longrightarrow \mathbb{F}_q[x]/\langle f_1\rangle \times \cdots \times \mathbb{F}_q[x]/\langle f_r\rangle = R_1 \times \cdots \times R_r.$$

Each R_i is a field with q^d elements and an algebraic extension of $\mathbb{F}_q$ of degree d:

$$\mathbb{F}_{q^d} \cong R_i = \mathbb{F}_q[x]/\langle f_i\rangle \supseteq \mathbb{F}_q.$$

We use the convention that for $a \in \mathbb{F}_q[x]$, we have $a \bmod f \in R$ and $\chi(a \bmod f) = (a \bmod f_1, \ldots, a \bmod f_r) = (\chi_1(a), \ldots, \chi_r(a))$, where $\chi_i(a) = a \bmod f_i \in R_i$. For $a \in \mathbb{F}_q[x]$ and $i \le r$, we have that f_i divides a if and only if $\chi_i(a) = 0$. If we obtain an $a \in \mathbb{F}_q[x]$ with some $\chi_i(a)$ equal to zero and others nonzero, then $\gcd(a, f)$ is a nontrivial divisor of f. We now describe a probabilistic procedure to find such a **splitting polynomial** a.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

We assume that q is odd, and write $e = (q^d - 1)/2$. For any $\beta \in R_i^\times = \mathbb{F}_{q^d}^\times$, we have $\beta^e \in \{1, -1\}$, and both possibilities occur equally often, by Lemma 14.7 with q^d instead of q. If we choose $a \in \mathbb{F}_q[x]$ with $\deg a < n$ and $\gcd(a, f) = 1$ uniformly at random, then $\chi_1(a), \ldots, \chi_r(a)$ are independent uniformly distributed elements of $\mathbb{F}_{q^d}^\times$, and $\varepsilon_i = \chi_i(a^e) \in R_i$ is 1 or -1, each with probability $1/2$. Therefore

$$\chi(a^e - 1) = (\varepsilon_1 - 1, \ldots, \varepsilon_r - 1),$$

and $a^e - 1$ is a splitting polynomial unless $\varepsilon_1 = \cdots = \varepsilon_r$. The latter occurs with probability $2 \cdot (1/2)^r = 2^{-r+1} \leq 1/2$.

ALGORITHM 14.8 Equal-degree splitting.

Input: A squarefree monic polynomial $f \in \mathbb{F}_q[x]$ of degree $n > 0$, where q is an odd prime power, and a divisor $d < n$ of n, so that all irreducible factors of f have degree d.

Output: A proper monic factor $g \in \mathbb{F}_q[x]$ of f, or "failure".

1. choose $a \in \mathbb{F}_q[x]$ with $\deg a < n$ at random
   if $a \in \mathbb{F}_q$ **then return** "failure"

2. $g_1 \longleftarrow \gcd(a, f)$
   if $g_1 \neq 1$ **then return** g_1

3. **call** the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $b = a^{(q^d-1)/2} \text{ rem } f$

4. $g_2 \longleftarrow \gcd(b - 1, f)$
   if $g_2 \neq 1$ and $g_2 \neq f$ **then return** g_2 **else return** "failure"

THEOREM 14.9.

Algorithm 14.8 works correctly as specified. It returns "failure" with probability less than $2^{1-r} \leq 1/2$, where $r = n/d \geq 2$, and takes $O((d \log q + \log n)\mathsf{M}(n))$ or $O^\sim(n^2 \log q)$ operations in $\mathbb{F}_q$.

PROOF. The failure probability has been given above as 2^{1-r} if $\gcd(a, f) = 1$. For general a, where step 2 might find a factor, the failure probability is less than 2^{1-r}. The cost for the gcds in steps 2 and 4 is $O(\mathsf{M}(n) \log n)$, and computing b in step 3 takes at most $2\log_2(q^d) \in O(d \log q)$ multiplications modulo f or $O(\mathsf{M}(n) d \log q)$ operations in $\mathbb{F}_q$. □

The usual trick of running the algorithm k times makes the failure probability less than $2^{(1-r)k} \leq 2^{-k}$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

EXAMPLE 14.5 (continued). We run Algorithm 14.8 on the remaining unfactored polynomial $f = x^4 + x^3 + x - 1 \in \mathbb{F}_3[x]$ from Example 14.5. We know that this polynomial factors into $r = 2$ irreducible polynomials of degree $d = 4/r = 2$. If our first choice in step 1 happens to be $a = x + 1$, then

$$\begin{aligned} g_1 &= \gcd(a, f) = \gcd(x+1, x^4 + x^3 + x - 1) = 1, \\ b &= a^4 \text{ rem } f = (x+1)^4 \text{ rem } x^4 + x^3 + x - 1 = -1, \\ g_2 &= \gcd(b-1, f) = \gcd(1, f) = 1, \end{aligned}$$

and this choice is unlucky. Our next random choice might be $a = x$. Then

$$\begin{aligned} g_1 &= \gcd(a, f) = \gcd(x, x^4 + x^3 + x - 1) = 1, \\ b &= a^4 \text{ rem } f = x^4 \text{ rem } x^4 + x^3 + x - 1 = -x^3 - x + 1, \\ g_2 &= \gcd(b-1, f) = \gcd(-x^3 - x, x^4 + x^3 + x^2 - 1) = x^2 + 1. \end{aligned}$$

The latter is one irreducible factor of f, and the other one is $f/(x^2+1) = x^2 + x - 1$.

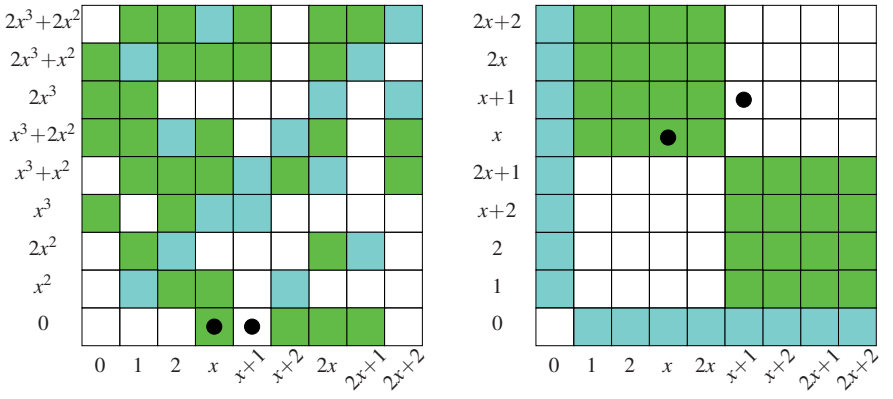


FIGURE 14.5: The lucky and unlucky choices for factoring $x^4 + x^3 + x - 1 \in \mathbb{F}_3[x]$.

Figure 14.5 illustrates the situation. On the left hand side, we have $R = F[x]/\langle f \rangle$, consisting of the 81 polynomials $a_3x^3 + a_2x^2 + a_1x + a_0 \bmod f$, with all $a_i \in \mathbb{F}_3$. The possible values for $a_1x + a_0$ are along the horizontal axis, and similarly $a_3x^3 + a_2x^2$ along the vertical axis. Our two choices are marked by a $\bullet$.

We have $R \cong \mathbb{F}_3[x]/\langle x^2+1 \rangle \times \mathbb{F}_3[x]/\langle x^2+x-1 \rangle \cong \mathbb{F}_9 \times \mathbb{F}_9$ on the right hand side, with the nine elements of the first factor on the horizontal axis, and the second factor on the vertical axis. We have arranged our two copies of $\mathbb{F}_9$ in an isomorphic way; in particular, mapping $x \bmod x^2+1$ to $x+2 \bmod x^2+x-1$ gives an isomorphism, since $(x+2)^2 + 1 \equiv 0 \bmod x^2 + x - 1$. On both axes, we first have 0, then the four nonzero squares, and then the four nonsquares.

The lucky choices of a are colored, the unlucky ones white. At right, it is clear what happens: the 16 blue elements with exactly one coordinate 0 give a

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

factorization in step 2, and the 32 green elements of type square/nonsquare or nonsquare/square are splitting polynomials. The $1 + 32 = 33$ other elements are unlucky. In general, only $2q^d - 2$ of the q^{rd} elements of R are "blue", so that for larger values of q^d it is very unlikely that the algorithm hits upon one of them by chance.

The representation at left is the one we can compute with. The magic of an isomorphism transforms this chaotic conglomerate into the disciplined diagram at right, about which we can easily reason, derive algorithms and prove their properties; these algorithms then are executed in the messy real world at left. $\diamond$

Algorithm 14.8 gives a factorization into two factors. If we need just one irreducible factor, we can apply the algorithm recursively to the smaller factor (Exercise 14.15). However, we will usually want all r factors, and this can be done by running the algorithm recursively on each factor.

ALGORITHM 14.10 Equal-degree factorization.

Input: A squarefree monic polynomial $f \in \mathbb{F}_q[x]$ of degree $n > 0$, for an odd prime power q, and a divisor d of n, so that all irreducible factors of f have degree d.

Output: The monic irreducible factors of f in $\mathbb{F}_q[x]$.

1. **if** $n = d$ **then return** f
2. **call** the equal-degree splitting algorithm 14.8 with input f and d repeatedly until it returns a proper factor $g \in \mathbb{F}_q[x]$ of f
3. **call** the algorithm recursively with input g and with input f/g
   return the results of the two recursive calls

THEOREM 14.11.

A squarefree polynomial of degree $n = rd$ with r irreducible factors of degree d can be completely factored with an expected number of $O((d \log q + \log n)\mathsf{M}(n) \log r)$ or $O^{\sim}(n^2 \log q)$ operations in $\mathbb{F}_q$.

PROOF. The workings of Algorithm 14.10 can be illustrated by means of a labeled tree (see Figure 14.6 for an example). The node labels are factors of f, with f at the root and the irreducible factors of f at the leaves. If the call to Algorithm 14.8 in step 2 returns "failure" at a particular node, then the corresponding node has precisely one child with the same label. Otherwise, it has two children labeled g and f/g. The product over all labels at one level of the tree is a divisor of f, and hence the total degree over all nodes at each level is at most n. The cost at a node of degree m is $O((d \log q + \log m)\mathsf{M}(m))$ operations in $\mathbb{F}_q$, by Theorem 14.9, and by the superlinearity of M (Section 8.3), the total cost at each level of the tree is $O((d \log q + \log n)\mathsf{M}(n))$ operations.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

We now show that the expected depth of the tree is $O(\log r)$, which together with $r \leq n$ implies the claims. Let $1 \leq i < j \leq r$ be fixed. Then in Algorithm 14.8, the probability that $a \bmod g_i$ and $a \bmod g_j$ are neither both squares nor both non-squares is at least $1/2$, by the Chinese Remainder Theorem. Thus for each level of the tree, the probability that a call to Algorithm 14.8 separates g_i and g_j at that level is at least $1/2$ (if they were not already separated before). Hence the probability that g_i and g_j are not yet separated at depth k is at most 2^{-k}. This is true for any pair of irreducible factors of f, and since there are $(r^2 - r)/2 \leq r^2$ such pairs, the probability p_k that not all irreducible factors are separated at depth k is at most $r^2 2^{-k}$. This is the probability that the depth of the tree is greater than k, and $p_{k-1} - p_k$ is the probability that the depth is exactly k. Let $s = \lceil 2 \log_2 r \rceil$. Then the expected depth of the tree is

$$\begin{aligned}\sum_{k \geq 1} k(p_{k-1} - p_k) = \sum_{k \geq 0} p_k &= \sum_{0 \leq k < s} p_k + \sum_{k \geq s} p_k \leq \sum_{0 \leq k < s} 1 + \sum_{k \geq s} r^2 2^{-k} \\ &= s + r^2 2^{-s} \sum_{k \geq 0} 2^{-k} \leq s + 2 \in O(\log r). \ \square\end{aligned}$$

EXAMPLE 14.12. Suppose that we want to find all the irreducible factors f_i of $f = f_0 \cdots f_9 \in \mathbb{F}_q[x]$, where the f_i are monic, irreducible, pairwise distinct, and have the same degree d.

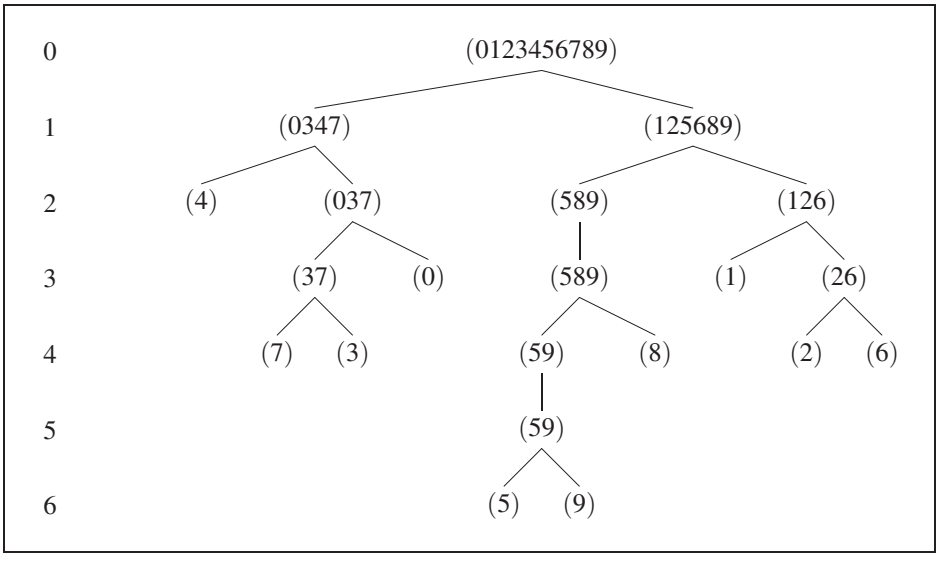


FIGURE 14.6: The workings of the equal-degree factorization algorithm 14.10 in Example 14.12.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

Figure 14.6 illustrates the process of a typical execution of Algorithm 14.10 in the form of a tree. The leaves correspond to the isolated irreducible factors. The labels are the indices of the irreducible factors of the polynomial at that node. The numbers in the left column are the levels. For example, the rightmost label at level 2 corresponds to the factor $f_1 f_2 f_6$ of f. The depth 6 is less than our upper bound $\lceil 2\log_2 10 \rceil + 2 = 9$ on the average value. $\diamond$

When q is large enough, there is a way to replace almost all powerings with exponent $(q^d - 1)/2$ in step 3 of Algorithm 14.8 by cheaper powerings with exponent $(q-1)/2$, leading to an expected time of $O(d\,\mathsf{M}(n)\log q + \mathsf{M}(n)\log(qn)\log r)$ operations in $\mathbb{F}_q$ for a variant of Algorithm 14.10 (Exercise 14.17).

14.4. A complete factoring algorithm

It remains to deal with polynomials that are not squarefree. Section 14.6 describes the squarefree factorization stage in detail, but we now discuss a simpler approach by modifying the distinct-degree factorization stage.

The algorithm proceeds as follows. For $i = 1, 2, \ldots$, the (squarefree) product g of all irreducible factors of f of degree i is computed, via one distinct-degree factorization step. Then g is factored into irreducibles, by calling the equal-degree factorization algorithm. For each irreducible factor g_j obtained, we determine its multiplicity e in f by trial division, and then remove g_j^e.

ALGORITHM 14.13 Polynomial factorization over finite fields.
Input: A nonconstant polynomial $f \in \mathbb{F}_q[x]$, where q is an odd prime power.
Output: The monic irreducible factors of f and their multiplicities.

1. $h_0 \longleftarrow x, \quad v_0 \longleftarrow \dfrac{f}{\mathrm{lc}(f)}, \quad i \longleftarrow 0, \quad U \longleftarrow \emptyset$
   repeat

2. $i \longleftarrow i + 1$
   { one distinct-degree factorization step }
   call the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $h_i = h_{i-1}^q \bmod f$
   $g \longleftarrow \gcd(h_i - x, v_{i-1})$

3. **if** $g \neq 1$ **then**
   { equal-degree factorization }
   call Algorithm 14.10 with input g and i to compute the monic irreducible factors $g_1, \ldots, g_s \in \mathbb{F}_q[x]$ of g

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

4. $v_i \longleftarrow v_{i-1}$
{ determine multiplicities }
for $j = 1, \ldots, s$ **do**
$e \longleftarrow 0$
while $g_j \mid v_i$ **do** $v_i \longleftarrow \dfrac{v_i}{g_j}, \quad e \longleftarrow e+1$
$U \longleftarrow U \cup \{(g_j, e)\}$

5. **until** $v_i = 1$

6. **return** U

As in the distinct-degree factorization algorithm 14.3, the algorithm may be aborted as soon as $\deg v_i < 2(i+1)$, and h_i need only be computed modulo v_{i-1} in step 2.

THEOREM 14.14.
Algorithm 14.13 correctly computes the irreducible factorization of f. If $\deg f = n$, then it takes an expected number of $O(n\mathsf{M}(n)\log(qn))$ or $O^{\sim}(n^2 \log q)$ arithmetic operations in $\mathbb{F}_q$.

PROOF. Let $f = \mathrm{lc}(f)\prod_{1\le i\le r} f_i^{e_i}$ be the irreducible factorization of f, with distinct monic irreducible polynomials $f_1, \ldots, f_r \in \mathbb{F}_q[x]$ and positive integers $e_1, \ldots, e_r$. We prove that the invariants

$$h_i \equiv x^{q^i} \bmod f, \quad v_i = \mathrm{lc}(f) \prod_{\deg f_k > i} f_k^{e_k}$$

hold each time before the algorithm passes through step 2. The first invariant is shown as in the proof of Theorem 14.4. The second one is clear for $i = 0$, and we may assume that $i \ge 1$. By Theorem 14.2, $x^{q^i} - x$ is the product of all distinct monic irreducible polynomials in $\mathbb{F}_q[x]$ of degree dividing i, and hence in particular it is squarefree. Thus, since $v_{i-1} \mid f$ and by the induction hypothesis, the polynomial

$$g = \gcd(h_i - x, v_{i-1}) = \gcd(x^{q^i} - x, v_{i-1}) = \prod_{\deg f_k = i} f_k$$

is a squarefree polynomial with irreducible factors of degree i only, and $g_1, \ldots, g_s$ are in fact the irreducible factors of g at the end of step 3 if $g \ne 1$. These are then removed with the correct multiplicity from v_i in step 4, and the invariants hold again before the next pass through step 2.

The cost for one execution of step 2 is $O(\mathsf{M}(n)\log(qn))$ operations in $\mathbb{F}_q$. There are at most n iterations of the outer loop, and hence the overall cost for step 2 is $O(n\mathsf{M}(n)\log(nq))$ operations. If $m_i = \deg g$, then step 3, the only probabilistic part

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

of the algorithm, takes an expected number of $O((i\log q+\log m_i)\mathsf{M}(m_i)\log(m_i/i))$ operations, by Theorem 14.11. Now

$$i\log(m_i/i) = m_i\frac{\log(m_i/i)}{m_i/i} \le m_i,$$

$$\begin{aligned}\sum_i (i\log q+\log m_i)\mathsf{M}(m_i)\log(m_i/i) &\le \sum_i (m_i\log q+\log^2 m_i)\mathsf{M}(n)\\ &\in O(n\mathsf{M}(n)\log q),\end{aligned}$$

where we have used $\sum_i m_i \le n$ and log is the binary logarithm. If e_j denotes the multiplicity of g_j in f, then one execution of the body of the **for** loop in step 4 takes $O(e_j\mathsf{M}(n))$ operations in $\mathbb{F}_q$, and the overall cost for step 4 is $O(n\mathsf{M}(n))$ operations, since the sum of the multiplicities of all irreducible factors of f is at most n. The timing estimate for step 2 is dominant, and the claim follows. □

We note that this is the same time bound as for the distinct-degree factorization algorithm 14.3 with a squarefree input.

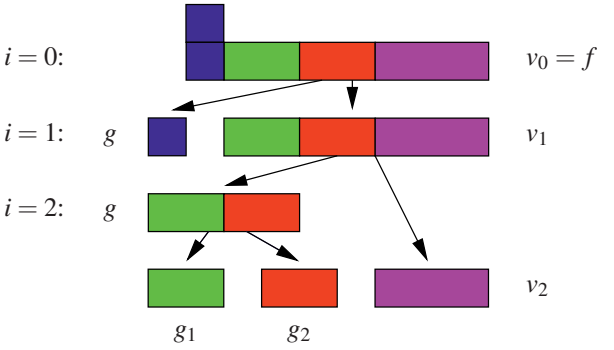


FIGURE 14.7: Sample polynomial factorization.

EXAMPLE 14.5 (continued). Let $q = 3$ and $f = x^9+x^8-x^7+x^6-x^4-x^3-x^2 \in \mathbb{F}_3[x]$ be the polynomial from Example 14.5 multiplied by x. Then Algorithm 14.13 computes $v_0 = f$ in step 1, and $h_1 = h_0^3 \text{ rem } f = x^3 \text{ rem } f = x^3$ and $g = \gcd(h_1 - x, f) = \gcd(x^3 - x, f) = x$ in step 2. The latter is obviously irreducible, so that $s = 1$ and $g_1 = g$ in step 3, and in step 4 we find that x divides f precisely twice, so that $(x,2)$ is adjoined to U and $v_1 = x^7+x^6-x^5+x^4-x^2-x-1$ at the end of step 4. For $i = 2$, we compute in step 2 that

$$\begin{aligned}h_2 &= h_1^3 \text{ rem } f = x^9 \text{ rem } f = -x^8+x^7-x^6+x^4+x^3+x^2,\\ g &= \gcd(h_2 - x, v_1) = x^4+x^3+x-1.\end{aligned}$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

In step 3, we find, as before, that the irreducible factors of g are $g_1 = x^2 + 1$ and $g_2 = x^2 + x - 1$. In step 4, we find that each of them has multiplicity one, adjoin $(g_1, 1)$ and $(g_2, 1)$ to U, and are left with $v_2 = x^3 - x + 1$. The algorithm would now continue for one further iteration, but the early abort condition $\deg v_2 < 2(2+1) = 6$ tells us that v_2 is already irreducible, and we are done. The trace is illustrated in Figure 14.7. $\diamond$

We now have the central result of this chapter: a complete factorization algorithm over finite fields in polynomial time. In the next sections, we study the problem in greater depth, discussing different (and faster) algorithms and various applications.

14.5. Application: root finding

If we are looking for all zeroes of a given polynomial $f \in \mathbb{F}_q[x]$, so that we want to determine all the linear factors of f, it is clearly sufficient to first compute $g = \gcd(x^q - x, f)$ and then apply the equal-degree factorization algorithm to g, so that we need not compute the *whole* distinct-degree decomposition of f.

ALGORITHM 14.15 Root finding over finite fields.
Input: A nonconstant polynomial $f \in \mathbb{F}_q[x]$.
Output: The distinct roots of f in $\mathbb{F}_q$.

1. **call** the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $h = x^q \text{ rem } f$
2. $g \longleftarrow \gcd(h - x, f), \quad r \longleftarrow \deg g$
   if $r = 0$ **then return** $\emptyset$
3. **call** the equal-degree factorization algorithm 14.10 to compute the irreducible factors $x - u_1, \ldots, x - u_r$ of g
4. **return** $u_1, \ldots, u_r$

COROLLARY 14.16.
Given $f \in \mathbb{F}_q[x]$ of degree n, we can find all roots of f in $\mathbb{F}_q$ using an expected number of $O(\mathsf{M}(n) \log n \log(nq))$ or $O^{\sim}(n \log q)$ operations in $\mathbb{F}_q$.

Algorithm 14.15 can be used to find all integral roots of a polynomial $f \in \mathbb{Z}[x]$ in a modular fashion, as follows.

ALGORITHM 14.17 Root finding over $\mathbb{Z}$ (big prime version).
Input: A nonconstant polynomial $f \in \mathbb{Z}[x]$ of degree n with max-norm $\|f\|_\infty = A$.
Output: The distinct roots of f in $\mathbb{Z}$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

1. $B \longleftarrow 2n(A^2+A)$
   let $p \in \mathbb{N}$ be an odd prime between $B+1$ and $2B$
2. **call** Algorithm 14.15 to find all distinct roots $\{u_1 \bmod p, \ldots, u_r \bmod p\}$ in $\mathbb{F}_p$ of $f \bmod p$, with $u_i \in \mathbb{Z}$ and $|u_i| < p/2$ for all i
3. **for** $i = 1, \ldots, r$ compute $v_i \in \mathbb{Z}[x]$ of degree $n-1$ and with max-norm less than $p/2$ such that $f \equiv (x-u_i)v_i \bmod p$
4. **return** $\{u_i : 1 \le i \le r,\ |u_i| \le A \text{ and } \|v_i\|_\infty \le nA\}$

THEOREM 14.18.
Algorithm 14.17 correctly computes all integral roots of f. The cost for step 2 is

$$O(\mathsf{M}(n)\log n \log(nA)\,\mathsf{M}(\log(nA))\operatorname{loglog}(nA))$$

or $O^\sim(n\log^2 A)$ word operations, and the cost for step 3 per u_i is $O(n\mathsf{M}(\log(nA)))$ or $O^\sim(n\log A)$ word operations.

PROOF. Dividing out powers of x if necessary, we may assume that $f(0) \neq 0$. If $f(u) = 0$ for some $u \in \mathbb{Z}$, then $(x-u) \mid f$ and u divides the constant coefficient of f, so that $|u| \le A < p/2$. Thus all distinct roots of f can be uniquely recovered from their images modulo p. We show that $f(u_i) = 0$ if and only if $|u_i| \le A$ and $\|v_i\|_\infty \le nA$, for all i. If $f(u_i) = 0$, then $|u_i| \le A$, by the above, and $\|f/(x-u_i)\|_\infty \le nA < p/2$, by Exercise 14.21. But $f/(x-u_i) \equiv v_i \bmod p$, and since both sides have coefficients less than $p/2$ in absolute value, they are equal. On the other hand, if $|u_i| \le A$ and $\|v_i\|_\infty \le nA$, then $\|(x-u_i)v_i\|_\infty \le (1+A)nA < p/2$, and the congruence $f \equiv (x-u_i)v_i \bmod p$ is in fact an equality.

We have $\log p \in O(\log(nA))$, and the cost estimate for step 2 follows from Corollaries 14.16 and 11.13. The cost for each u_i in step 3 is $O(n)$ additions and multiplications in $\mathbb{F}_p$ or $O(n\mathsf{M}(\log(nA)))$ word operations. □

The cost for finding p is discussed in Section 18.4. In Section 15.6, we will discuss a faster algorithm for computing integer roots.

14.6. Squarefree factorization

For the moment, we let F be an arbitrary field. We will show how to reduce the problem of factoring arbitrary polynomials to that of factoring squarefree polynomials. Polynomial factorization software usually performs this reduction first; it is not quite clear when the (small) advantage one gains outweighs the (small) cost of the reduction. Our purpose in skipping it was to describe, with Algorithm 14.13, a factorization method that is conceptionally as simple as possible.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

In Section 9.3, we defined the derivative f' of a polynomial $f = \sum_{0\le i\le n} a_i x^i \in F[x]$ as $f' = \partial f/\partial x = \sum_{0\le i\le n} i a_i x^{i-1}$.

Suppose that f is not squarefree, so that $f = g^2 h$ for some $g, h \in F[x]$ and $g \notin F$. Then $f' = g\cdot(2g'h + gh')$, so that g divides $u = \gcd(f, f')$. If the characteristic (Section 25.3) of F is zero, then $f' \neq 0$ and $\deg f' < \deg f = n$, so that u is a proper divisor of f.

Now let $f = \prod_{1\le i\le r} f_i^{e_i}$ be the irreducible factorization of the monic polynomial $f \in F[x]$, with distinct monic irreducible $f_1, \ldots, f_r$ and positive $e_1, \ldots, e_r \in \mathbb{N}$. We define the **squarefree part** of f to be $\prod_{1\le i\le r} f_i$. Then

$$f' = \sum_{1\le i\le r} e_i \frac{f}{f_i} f_i' \tag{2}$$

(Exercise 14.22). In order to compute the squarefree part, let us determine $u = \gcd(f, f')$. The only possible divisors are powers of $f_1, \ldots, f_r$. But what is the multiplicity of f_i in f'? Each summand in (2) but the ith is divisible by $f_i^{e_i}$, and the ith equals $e_i f_i' \cdot f/f_i$ and is divisible by $f_i^{e_i-1}$. Thus $f_i^{e_i-1}$ divides f'. Can $f_i^{e_i}$ divide it? This happens if and only if it divides $e_i f_i' \cdot f/f_i$. Certainly f_i divides none of the other factors in f/f_i, except $f_i^{e_i-1}$, but can it divide $e_i f_i'$? This is a polynomial of degree smaller than $\deg f_i$, and one might think that it cannot be divisible by f_i. But indeed it can, namely when (and only when) $e_i f_i' = 0$. We leave this situation to Exercises 14.27 and 14.30, and now put together our current knowledge in the case where this cannot happen, namely when the characteristic is zero.

ALGORITHM 14.19 Squarefree part in characteristic zero.

Input: $f \in F[x]$ monic of degree $n > 0$, where F is a field of characteristic zero.

Output: The squarefree part of f, that is, the product of all distinct monic irreducible factors of f.

1. $u \longleftarrow \gcd(f, f')$
2. **return** $v = \dfrac{f}{u}$.

THEOREM 14.20.

Algorithm 14.19 works correctly as specified and takes $O(\mathsf{M}(n)\log n)$ operations in F.

PROOF. For the correctness, we note that each $e_i f_i'$ is nonzero, and by the above

$$u = \prod_{1\le i\le r} f_i^{e_i-1}, \quad v = \prod_{1\le i\le r} f_i.$$

The running time estimate follows from Theorems 9.6 and 11.7. □

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

The **squarefree decomposition** provides more information than the squarefree part. For a nonconstant monic polynomial $f \in F[x]$, this is the unique sequence of monic squarefree pairwise coprime polynomials $(g_1, \ldots, g_m)$ with

$$f = g_1 g_2^2 g_3^3 \cdots g_m^m$$

and $g_m \neq 1$. For example, the squarefree decomposition of $x^4(x+1)^2(x-1)^2 \cdot (x^2+1)^2(x^2+x+1) \in \mathbb{Q}[x]$ is $(x^2+x+1, x^4-1, 1, x)$. Thus g_i is the product of those monic irreducible polynomials that divide f exactly i times. The squarefree part of f is $g_1 \cdots g_m$. The process of computing the squarefree decomposition is called **squarefree factorization**.

We now present algorithms for computing the squarefree decomposition. This decomposition will be used in an integration algorithm in Chapter 22. For simplicity, we assume characteristic zero. A simple idea is to call Algorithm 14.19, compute $g_1 = v/\gcd(u,v)$, and proceed recursively with f replaced by u. This takes $O(m\mathsf{M}(n)\log n)$ field operations. The following method is faster by one order of magnitude.

ALGORITHM 14.21 Yun's squarefree factorization in characteristic zero.
Input: A monic polynomial $f \in F[x]$ of degree $n \geq 1$, where F is a field of characteristic zero.
Output: The squarefree decomposition of f.

1. $u \longleftarrow \gcd(f, f')$, $\quad v_1 \longleftarrow \dfrac{f}{u}$, $\quad w_1 \longleftarrow \dfrac{f'}{u}$
2. $i \longleftarrow 1$
   repeat
   $\quad h_i \longleftarrow \gcd(v_i, w_i - v_i')$, $\quad v_{i+1} \longleftarrow \dfrac{v_i}{h_i}$, $\quad w_{i+1} \longleftarrow \dfrac{w_i - v_i'}{h_i}$
   $\quad i \longleftarrow i+1$
   until $v_i = 1$
   $k \longleftarrow i-1$
3. **return** $(h_1, \ldots, h_k)$

The correctness of the algorithm depends on the following lemma.

LEMMA 14.22. *Let F be a field of characteristic zero, $g_1, \ldots, g_m \in F[x]$ monic squarefree and pairwise coprime, $g = g_1 \cdots g_m$, and $h = \sum_{1 \leq i \leq m} c_i g_i' g/g_i$, for some constants $c_i \in F$. Then $\gcd(g, h - cg') = \prod_{c_j = c} g_j$ for all $c \in F$.*

PROOF. We have $g' = \sum_{1 \leq i \leq m} g_i' g/g_i$, by Exercise 9.27, and hence

$$h - cg' = \sum_{1 \leq i \leq m} (c_i - c) g_i' \frac{g}{g_i}.$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

Now g_j divides each summand with $i \neq j$, and $\gcd(g_j, g_j') = \gcd(g_j, g/g_j) = 1$ since F has characteristic zero and g_j and g are squarefree. Thus the claim follows from

$$\gcd(g_j, h - cg') = \gcd\left(g_j, (c_j - c)g_j' \frac{g}{g_j}\right) = \gcd(g_j, c_j - c). \square$$

THEOREM 14.23.
The algorithm uses $O(\mathsf{M}(n)\log n)$ operations in F and it computes correctly the squarefree decomposition of f.

PROOF. Let $(g_1, \ldots, g_m)$ be the squarefree decomposition of f. Then we have $u = g_2 g_3^2 \cdots g_m^{m-1}$, and we prove by induction on i that

$$h_i = g_i \text{ if } i \geq 1, \quad v_{i+1} = \prod_{i<j\leq m} g_j, \quad w_{i+1} = \sum_{i<j\leq m} (j-i) g_j' \frac{v_{i+1}}{g_j}$$

for $0 \leq i \leq m$. This is clear for v_1, and the claim for w_1 follows from

$$f' = \sum_{1\leq j\leq m} \frac{f}{g_j} \cdot j g_j' = u \sum_{1\leq j\leq m} j \frac{v_1}{g_j} g_j'.$$

For $i \geq 1$, Lemma 14.22 gives $h_i = g_i$. Then $v_{i+1} = \prod_{i<j\leq m} g_j$ is clear, and

$$\begin{aligned} w_{i+1} &= \left(\sum_{i\leq j\leq m} (j - (i-1)) g_j' \frac{v_i}{g_j} - \sum_{i\leq j\leq m} g_j' \frac{v_i}{g_j} \right) / g_i \\ &= \sum_{i<j\leq m} (j-i) g_j' \frac{v_i}{g_j g_i} = \sum_{i<j\leq m} (j-i) g_j' \frac{v_{i+1}}{g_j}. \end{aligned}$$

For the cost estimate, let $d_j = \deg g_j$ for $1 \leq j \leq m$. Step 1 takes $O(\mathsf{M}(n)\log n)$ arithmetic operations. Moreover, $\deg v_i = \sum_{i\leq j\leq m} d_j$, $\deg w_i = (\deg v_i) - 1$, the gcd computation in the ith loop takes $O(\mathsf{M}(\deg v_i)\log n)$, and the two division steps $O(\mathsf{M}(\deg v_i))$ operations in F. Using the superlinearity of M (Section 8.3), we find

$$\begin{aligned} \sum_{1\leq i\leq m} \mathsf{M}(\deg v_i) &\leq \mathsf{M}\Big(\sum_{1\leq i\leq m} \deg v_i \Big) = \mathsf{M}\Big(\sum_{1\leq i\leq j\leq m} d_j \Big) \\ &= \mathsf{M}\Big(\sum_{1\leq i\leq m} i d_i \Big) = \mathsf{M}(n). \square \end{aligned}$$

EXAMPLE 14.24. Suppose that $f = abc^2d^4$ for monic distinct irreducible polynomials $a, b, c, d \in F[x]$. Then Algorithm 14.21 computes $u = \gcd(f, f') = cd^3$,

$$\begin{aligned} v_1 &= f/u = abcd, \quad w_1 = f'/u = a'bcd + ab'cd + 2abc'd + 4abcd', \\ h_1 &= \gcd(abcd, abc'd + 3abcd') = ab, \end{aligned}$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

$$v_2 = abcd/ab = cd, \quad w_2 = (abc'd + 3abcd')/ab = c'd + 3cd',$$
$$h_2 = \gcd(cd, 2cd') = c, \quad v_3 = cd/c = d, \quad w_3 = 2cd'/c = 2d',$$
$$h_3 = \gcd(d, d') = 1, \quad v_4 = d/1 = d, \quad w_4 = d'/1 = d',$$
$$h_4 = \gcd(d, 0) = d, \quad v_5 = d/d = 1, \quad w_5 = 0/d = 0.$$

Thus the correct squarefree decomposition $(ab, c, 1, d)$ is returned. $\diamond$

An interesting possibility may occur if $\operatorname{char} F = p$ for a prime p, which does not happen if $\operatorname{char} F = 0$: $f = \sum_{0 \le i \le n} a_i x^i \notin F$ and $f' = 0$. This happens if and only if each i with $a_i \neq 0$ is divisible by p; then the summand $i a_i x^{i-1}$ is zero in $F[x]$. If $F = \mathbb{F}_p$, then we can write

$$f = \sum_{0 \le i \le n/p} a_{ip} x^{ip} = \left(\sum_{0 \le i \le n/p} a_{ip} x^i \right)^p, \tag{3}$$

since $(g+h)^p = g^p + h^p$ for all $g, h \in \mathbb{F}_p[x]$ and $a_{ip}^p = a_{ip}$ for all $a_{ip} \in \mathbb{F}_p$ (Section 25.4). For example, $(x^4 + x^2 + 1)' = 0$ in $\mathbb{F}_2[x]$, and $x^4 + x^2 + 1 = (x^2 + x + 1)^2$.

Similarly, if $F = \mathbb{F}_q$ for a prime power $q = p^s$ and $s \ge 1$, then Fermat's little theorem 14.1 says that $a^q = a$ for all $a \in \mathbb{F}_q$, and hence $a^{p^{s-1}} = a^{q/p}$ is a pth root of a. Then for $g = \sum_{0 \le i \le n/p} a_{ip}^{q/p} x^i$, we have $f = g^p$, in analogy to (3). On the other hand, if $f = g^p$, then $f' = p g^{p-1} g' = 0$, and thus

$$\forall f \in \mathbb{F}_q[x] \quad f' = 0 \iff f \text{ is a } p\text{th power in } \mathbb{F}_q[x]. \tag{4}$$

Let $f = f_1^{e_1} \cdots f_r^{e_r}$ be the irreducible factorization of f. For $1 \le i \le r$, $f_i' = 0$ would imply that f_i is a pth power, in contradiction to the irreducibility of f_i. In the language of field theory, any irreducible polynomial has a nonzero derivative and thus is **separable**, and therefore finite fields and fields of characteristic zero are **perfect**. The field $F = \mathbb{F}_p(y)$ of rational functions in y is not perfect, since for example $f = x^p - y \in F[x]$ is irreducible with $f' = 0$ (Exercise 14.33). The reason for this is that the "constant" $y \in F$ has no pth root in F. Thus $f_i' \neq 0$ over a finite field, and $\deg f_i' < \deg f_i$ implies that the $\gcd(f_i', f_i)$ is not f_i and hence 1, by the irreducibility of f_i. But it can still happen that $e_i f_i' = 0$, namely when the nonzero integer e_i is divisible by p. In that case, (2) implies that $f_i^{e_i}$ divides f'.

COROLLARY 14.25.
Let F be a finite field or a field of characteristic zero and $f \in F[x]$ nonconstant. Then f is squarefree if and only if $\gcd(f, f') = 1$.

Exercises 14.27 and 14.30 discuss squarefree factorization over finite fields.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

14.7. The iterated Frobenius algorithm

We now sketch a faster algorithm for our problem of polynomial factorization. A central role is played by the **Frobenius automorphism**

$$\sigma: \begin{cases} \mathbb{F}_{q^n} & \longrightarrow & \mathbb{F}_{q^n}, \\ \alpha & \longmapsto & \alpha^q. \end{cases}$$

In the language of Galois theory, the field extension $\mathbb{F}_{q^n}/\mathbb{F}_q$ is normal, σ generates its Galois group $\{\mathrm{id}, \sigma, \ldots, \sigma^{n-1}\}$, and $\sigma^n = \mathrm{id}$, by Fermat's little theorem 14.1.

For a squarefree monic polynomial $f \in \mathbb{F}_q[x]$ of degree n, we now consider the residue class ring $R = \mathbb{F}_q[x]/\langle f \rangle$ and the map

$$\sigma: \begin{cases} R & \longrightarrow & R, \\ \alpha & \longmapsto & \alpha^q, \end{cases}$$

which is also called the **Frobenius automorphism** of R (it is in fact an automorphism since f is squarefree). It generalizes the above notion, where $R = \mathbb{F}_{q^n}$ if f is irreducible, and satisfies the following rules for all $\alpha, \beta \in R$:

$$\sigma(\alpha+\beta) = \sigma(\alpha)+\sigma(\beta), \quad \sigma(\alpha\beta) = \sigma(\alpha)\sigma(\beta), \quad \sigma(\alpha) = \alpha \iff \alpha \in \mathbb{F}_q.$$

The last one is Fermat's little theorem, where we identify $\mathbb{F}_q$ with the subfield of R consisting of the residue classes of the constant polynomials modulo f. These rules imply in particular that $g(\alpha^q) = g(\alpha)^q$ for any $\alpha \in R$ and any polynomial $g \in \mathbb{F}_q[x]$.

Let $\xi = (x \bmod f) \in R$. Then the powers $1, \xi, \cdots, \xi^{n-1}$ of ξ form an $\mathbb{F}_q$-basis of R, and any element $\alpha \in R$ can be written as

$$\alpha = a_{n-1}\xi^{n-1} + \cdots + a_1\xi + a_0 = a(\xi) = (a \bmod f),$$

with unique $a_{n-1}, \ldots, a_0 \in \mathbb{F}_q$, where $a = a_{n-1}x^{n-1} + a_1 x + a_0 \in \mathbb{F}_q[x]$ is the canonical representative in $\mathbb{F}_q[x]$ of degree less than n for the residue class α. We will write $\breve{\alpha}$ for a in what follows, so that

$$\breve{\alpha}(\xi) = \breve{\alpha} \bmod f = \alpha. \tag{5}$$

In software, there is no need to distinguish between α and $\breve{\alpha}$, or, equivalently, between $(a \bmod f)$ and a. Both will be represented by an array $[a_0, \ldots, a_{n-1}]$ of coefficients. But conceptually, one has to make the distinction; for example, it does not make sense to "evaluate α at ξ^q", but we can (and will) evaluate $\breve{\alpha}$ at ξ^q.

If $\alpha \in R$ is arbitrary and $a = \breve{\alpha} \in \mathbb{F}_q[x]$ as above, we can compute the image of α under the Frobenius map as

$$\alpha^q = a(\xi)^q = a(\xi^q) = \breve{\alpha}(\xi^q). \tag{6}$$

This is called the **polynomial representation** of the Frobenius map and is the basis for the following algorithm.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

ALGORITHM 14.26 Iterated Frobenius.
Input: $f \in \mathbb{F}_q[x]$ squarefree of degree n, $d \in \mathbb{N}$ with $d \le n$, $\xi^q \in R = \mathbb{F}_q[x]/\langle f \rangle$, where $\xi = x \bmod f \in R$, and $\alpha \in R$.
Output: $\alpha, \alpha^q, \ldots, \alpha^{q^d} \in R$.

1. $\gamma_0 \longleftarrow \xi, \quad \gamma_1 \longleftarrow \xi^q, \quad l \longleftarrow \lceil \log_2 d \rceil$
2. **for** $i = 1, \ldots, l$ **do**
   call the fast multipoint evaluation algorithm 10.7 over R to compute $\gamma_{2^{i-1}+j} = \check{\gamma}_{2^{i-1}}(\gamma_j)$ for $1 \le j \le 2^{i-1}$
3. **call** the fast multipoint evaluation algorithm 10.7 over R to compute $\delta_k = \check{\alpha}(\gamma_k)$ for $0 \le k \le d$
4. **return** $\delta_0, \ldots, \delta_d$

We note that the input ξ^q is not required for the correctness of the algorithm, but we need it for the running time bound.

THEOREM 14.27.
Algorithm 14.26 works correctly as specified and uses $O(\mathsf{M}(n)^2 \log n \log d)$ or $O^{\sim}(n^2)$ operations in $\mathbb{F}_q$.

PROOF. For the correctness, we prove the invariant $\gamma_k = \xi^{q^k}$ for $0 \le k \le 2^i$ by induction on i. The case $i = 0$ is clear from step 1. For the inductive step, it is sufficient to prove the claim for $k > 2^{i-1}$. For $1 \le j \le 2^{i-1}$, we have that

$$\gamma_{2^{i-1}+j} = \check{\gamma}_{2^{i-1}}(\gamma_j) = \check{\gamma}_{2^{i-1}}(\xi^{q^j}) = (\check{\gamma}_{2^{i-1}}(\xi))^{q^j} = \gamma_{2^{i-1}}^{q^j} = \left(\xi^{q^{2^{i-1}}}\right)^{q^j} = \xi^{q^{2^{i-1}+j}},$$

by step 2, (5), (6), and the induction hypothesis. Finally, in step 3 we correctly compute

$$\delta_k = \check{\alpha}(\gamma_k) = \check{\alpha}(\xi^{q^k}) = \check{\alpha}(\xi)^{q^k} = \alpha^{q^k}$$

for $0 \le k \le d$.

By Corollary 10.8, the polynomials $\check{\gamma}_{2^{i-1}}$ and $\check{\alpha}$ in $R[x]$ of degree less than n can be evaluated at no more than n ring elements using at most $(\frac{11}{2}\mathsf{M}(n) + O(n)) \log_2 n$ multiplications and additions in R. In steps 2 and 3 of Algorithm 14.26, we solve $l + 1 \in O(\log d)$ such multipoint evaluation problems, at a total cost of $O(\mathsf{M}(n) \log n \log d)$ operations in R, or $O(\mathsf{M}(n)^2 \log n \log d)$ operations in $\mathbb{F}_q$. □

Exercise 14.35 asks for a better O-free timing estimate. In fact,

$$O((n/d)\mathsf{M}(nd) \log d) \tag{7}$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

operations in $\mathbb{F}_q$ are sufficient, but since this saves only factors $\log n$ when $d = n$, we omit the proof.

The process of the iterated Frobenius algorithm can be illustrated as follows:

$$\xi^{q^1} \underbrace{\xi^{q^2}}_{i=1} \underbrace{\xi^{q^3}\ \xi^{q^4}}_{i=2} \underbrace{\xi^{q^5}\ \xi^{q^6}\ \xi^{q^7}\ \xi^{q^8}}_{i=3} \underbrace{\xi^{q^9}\ \xi^{q^{10}}\ \xi^{q^{11}}\ \xi^{q^{12}}\ \xi^{q^{13}}\ \xi^{q^{14}}\ \xi^{q^{15}}\ \xi^{q^{16}}}_{i=4} \ \dots$$

The ith brace encloses those powers of ξ that are newly computed in the ith iteration of step 2. The advantage of the iterated Frobenius algorithm over the naïve successive computation of the ξ^{q^k} might be compared to the advantage of repeated squaring for the computation of one power a^n over repeated multiplication.

Algorithm 14.26 can be used for distinct-degree factorization as well as for equal-degree factorization. Remember that in Algorithm 14.3, we had to compute

$$x^{q^i} - x \bmod f = \xi^{q^i} - \xi = \gamma_i - \xi$$

for $1 \le i \le n$. This can be done by first computing ξ^q using repeated squaring and then applying steps 1 and 2 of the iterated Frobenius algorithm with $d = n$. The cost for the other steps in the distinct-degree factorization algorithm is dominated by the cost for the iterated Frobenius, and using (7), we have the following corollary.

COROLLARY 14.28.
The distinct-degree decomposition of a squarefree polynomial $f \in \mathbb{F}_q[x]$ of degree n can be computed using $O(\mathsf{M}(n^2)\log n + \mathsf{M}(n)\log q)$ or $O^\sim(n^2 + n\log q)$ operations in $\mathbb{F}_q$.

In the equal-degree factorization of Algorithm 14.8, we compute $\alpha^{(q^d-1)/2}$ for a uniform random element $\alpha \in R = \mathbb{F}_q[x]/\langle f\rangle$, where d is the degree of any of the irreducible factors of f. The exponent can be written as

$$\frac{q^d - 1}{2} = (q^{d-1} + q^{d-2} + \cdots + q + 1)\frac{q-1}{2},$$

and hence

$$\alpha^{(q^d-1)/2} = (\alpha^{q^{d-1}} \cdots \alpha^q \cdot \alpha)^{(q-1)/2} = (\delta_{d-1} \cdots \delta_1 \cdot \delta_0)^{(q-1)/2},$$

which can be computed using the iterated Frobenius algorithm, and repeated squaring for the computation of the initial power ξ^q and the final $\frac{q-1}{2}$th power. Again using (7), we have the following result.

COROLLARY 14.29.
The complete factorization of a squarefree polynomial $f \in \mathbb{F}_q[x]$ of degree $n = rd$ with r irreducible factors of degree d can be computed using an expected number of $O((\mathsf{M}(nd)r\log d + \mathsf{M}(n)\log q)\log r)$ or $O^\sim(n^2 + n\log q)$ operations in $\mathbb{F}_q$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

Similarly as for the equal-degree factoring algorithm 14.10, one finds a slightly better estimate of $O(\mathsf{M}(nd)r\log d + \mathsf{M}(n)\log r\log q)$ arithmetic operations in $\mathbb{F}_q$ (Exercise 14.17), or even $O(\mathsf{M}(nd)r + \mathsf{M}(n)\log q)$ for finding only one irreducible factor. By replacing steps 2 and 3 in Algorithm 14.13 with the two algorithms for distinct-degree and equal-degree factorization presented above, we obtain the following result.

COROLLARY 14.30.
A polynomial $f \in \mathbb{F}_q[x]$ of degree n can be completely factored with an expected number of $O(\mathsf{M}(n^2)\log n + \mathsf{M}(n)\log n\log q)$ or $O^{\sim}(n^2 + n\log q)$ operations in $\mathbb{F}_q$.

A single operation in $\mathbb{F}_q$ can be done with $O(\mathsf{M}(\log q)\operatorname{loglog} q)$ or $O^{\sim}(\log q)$ word operations. Factorization thus takes $O^{\sim}(n^2\log q + n\log^2 q)$ word operations, which is softly quadratic in the input size of about $n\log_{2^{64}} q$ words. In the Notes 14.8 we mention subquadratic algorithms by Kaltofen & Shoup (1998) in the case of small characteristic.

14.8. Algorithms based on linear algebra

The earliest factoring algorithm over finite fields running in polynomial time is due to Berlekamp (1967, 1970). Instead of performing a distinct-degree factorization, he takes a linear algebra approach to split the polynomial, as follows. Let $f \in \mathbb{F}_q[x]$ be a squarefree monic polynomial of degree $n > 0$ and $R = \mathbb{F}_q[x]/\langle f\rangle$. Then R is a vector space of dimension n over $\mathbb{F}_q$ (in fact, it is even an $\mathbb{F}_q$-algebra), and the map $\beta = \sigma - \mathrm{id}: R \longrightarrow R$ with $\beta(a) = a^q - a$ is $\mathbb{F}_q$-linear. Let us determine its kernel. If $f = f_1 \cdots f_r$ is the factorization of f into distinct monic irreducible polynomials $f_1, \ldots, f_r \in \mathbb{F}_q[x]$, then we have the Chinese remainder decomposition

$$R \cong \mathbb{F}_q[x]/\langle f_1\rangle \times \cdots \times \mathbb{F}_q[x]/\langle f_r\rangle. \tag{8}$$

As in Section 14.3, each $\mathbb{F}_q[x]/\langle f_i\rangle$ is a finite field with $q^{\deg f_i}$ elements and contains $\mathbb{F}_q$ as a subfield (the constant polynomials modulo f_i). Now for $a \in \mathbb{F}_q[x]$, we have

$$\begin{aligned} a \bmod f \in \ker\beta &\iff a^q \equiv a \bmod f \iff a^q \equiv a \bmod f_i \text{ for } 1 \le i \le r \\ &\iff a \bmod f_i \in \mathbb{F}_q \text{ for } 1 \le i \le r, \end{aligned}$$

by Fermat's little theorem 14.1. Thus $\mathcal{B} = \ker\beta$ is the subspace corresponding to $\mathbb{F}_q \times \cdots \times \mathbb{F}_q = \mathbb{F}_q^r$ in (8), as illustrated in Figure 14.8. In fact, $\mathcal{B}$ is an $\mathbb{F}_q$-subalgebra of R, called the **Berlekamp subalgebra**. If χ is the isomorphism in (8), then $a \bmod f \in \mathcal{B}$ if and only if $\chi(a \bmod f) = (a_1 \bmod f_1, \ldots, a_r \bmod f_r)$ for some constants $a_1, \ldots, a_r \in \mathbb{F}_q$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

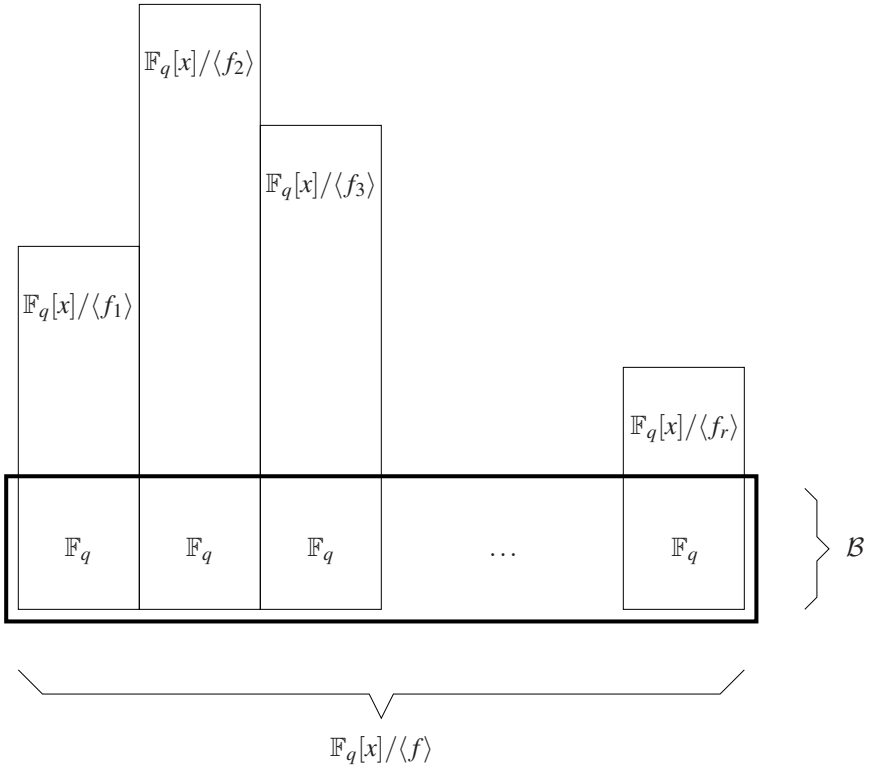


FIGURE 14.8: The Berlekamp subalgebra $\mathcal{B}$ of $R = \mathbb{F}_q[x]/\langle f\rangle$.

The matrix $Q \in \mathbb{F}_q^{n\times n}$ representing the Frobenius map σ with respect to the polynomials basis $x^{n-1} \bmod f, \ldots, x \bmod f, 1 \bmod f$ of R was first used in Petr (1937) for distinct-degree factorization, and has been a staple of computer algebra since Berlekamp's work. We will call it the **Petr-Berlekamp matrix** of f. Now Berlekamp's algorithm first determines a basis $b_1 \bmod f, \ldots, b_r \bmod f$ of $\mathcal{B}$ using Gaussian elimination on $Q - I$. We note that

$$f \text{ is irreducible} \iff r = 1 \iff \operatorname{rank}(Q - I) = n - 1. \tag{9}$$

Now we assume for simplicity that q is odd (see Exercise 14.16 for characteristic 2), and let $b = c_1 b_1 + \cdots + c_r b_r$ be a uniformly random linear combination of basis elements, with $c_1, \ldots, c_r \in \mathbb{F}_q$ chosen independently, so that $b \bmod f$ is a uniform random element of $\mathcal{B}$. We now employ the same $(q-1)/2$ trick as in the equal-degree factorization. The $b \bmod f_i$ are uniformly and independently distributed random elements of $\mathbb{F}_q$ for $1 \le i \le r$. Hence, if no f_i divides b, then $b^{(q-1)/2} \equiv \pm 1 \bmod f_i$, and both possibilities occur with probability $1/2$ and independently for all i, by Lemma 14.7. This yields the following Las Vegas algorithm.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

ALGORITHM 14.31 Berlekamp's algorithm.
Input: A monic squarefree polynomial $f \in \mathbb{F}_q[x]$ of degree $n > 0$, where q is an odd prime power.
Output: Either a proper factor g of f, or "failure".

1. **call** the repeated squaring algorithm 4.8 in $\mathbb{F}_q[x]/\langle f \rangle$ to compute x^q rem f
2. **for** $i = 0, \ldots, n-1$ compute x^{qi} rem $f = \sum_{0 \le j < n} q_{ij} x^j$
   $Q \longleftarrow (q_{ij})_{0 \le i,j < n}$
3. use Gaussian elimination on $Q - I \in \mathbb{F}_q^{n \times n}$, where I is the $n \times n$ identity matrix, to compute the dimension $r \in \mathbb{N}$ and a basis $b_1 \bmod f, \ldots, b_r \bmod f$ of the Berlekamp algebra $\mathcal{B}$, with $b_1, \ldots, b_r \in \mathbb{F}_q[x]$ of degree less than n
   if $r = 1$ **then return** f
4. choose independent uniformly random elements $c_1, \ldots, c_r \in \mathbb{F}_q$
   $a \longleftarrow c_1 b_1 + \cdots + c_r b_r$
5. $g_1 \longleftarrow \gcd(a, f)$
   if $g_1 \neq 1$ and $g_1 \neq f$ **then return** g_1
6. **call** the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $b = a^{(q-1)/2}$ rem f
7. $g_2 \longleftarrow \gcd(b - 1, f)$
   if $g_2 \neq 1$ and $g_2 \neq f$ **then return** g_2 **else return** "failure"

The Gaussian elimination in step 3 can be done with $O(n^3)$ operations in $\mathbb{F}_q$. In Section 12.1 we have seen faster methods, and we may use any feasible matrix multiplication exponent ω, so that $n \times n$ matrices can be multiplied with $O(n^\omega)$ operations. The reader may very well think of the classical $\omega = 3$.

THEOREM 14.32.
Algorithm 14.31 works correctly as specified and returns "failure" with probability at most $1/2$. *It uses* $O(n^\omega + \mathsf{M}(n) \log q)$ *operations in* $\mathbb{F}_q$ *if* $\omega > 2$.

PROOF. Correctness is clear from the discussion preceding the algorithm. In order to analyze the failure probability, we note that a is a uniformly random element of $\mathcal{B}$, so that $u_i \equiv a \bmod f_i$ for $1 \le i \le r$ are independent random elements of $\mathbb{F}_q$ (via its embedding in $\mathbb{F}_q[x]/\langle f \rangle$). If some u_i is zero and some u_j nonzero, a factor is returned in step 5. With probability q^{-r}, all u_i's are zero. All u_i's are nonzero with probability $(1 - q^{-1})^r$, and then each $v_i = u_i^{(q-1)/2}$ is 1 or -1 with probability 2^{-1} for either case, and all v_i's are equal with probability $2 \cdot 2^{-r}$. Thus failure occurs

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

in step 7 with probability $t = q^{-r} + (1-q^{-1})^r \cdot 2^{-r+1} < 2^{-1}$, since $r \geq 2$, the last inequality holds for $r = 2$, and t is monotonically decreasing in r.

The cost for step 1 is $O(\mathsf{M}(n)\log q)$ field operations. Step 2 uses $n-2$ multiplications modulo f, or $O(n\mathsf{M}(n))$ operations in $\mathbb{F}_q$. The cost for step 3 is $O(n^\omega)$, by Section 12.1. This dominates the cost for step 2, the $O(nr)$ field operations for step 4, and the $O(\mathsf{M}(n)\log n)$ for the gcds in steps 5 and 7. Finally, step 6 uses another $O(\mathsf{M}(n)\log q)$ field operations. □

When we want the complete factorization of f, it is sufficient to compute the basis of $\mathcal{B}$ only once and to apply the splitting process of steps 4 through 7 recursively to g and f/g, as in the equal-degree factorization algorithm 14.10. Then an analysis similar to that of Theorem 14.11 implies that all irreducible factors of f can be computed in expected time $O(n^\omega + \mathsf{M}(n)\log r \log q)$.

When $\log q$ is small in comparison to n, then the dominant cost of Algorithm 14.31 is the nullspace computation in step 2. Since evaluating the matrix $Q - I$ at a vector corresponds to computing $\beta(a) = a^q - a$ for some $a \in R$, at a cost of $O(\mathsf{M}(n)\log q)$ field operations, it looks promising to try the black box linear algebra approach from Section 12.4. By combining the algorithm of Kaltofen & Saunders (1991) (see Theorem 12.19 and the discussion following it) with Berlekamp's algorithm in a straightforward manner, we would obtain an $O^\sim(n^2 \log q)$ factoring algorithm. Exploiting the special properties of the Frobenius automorphism, Kaltofen & Lobo (1994) have taken the following more sophisticated approach which leads to an $O^\sim(n^2 + n\log q)$ algorithm.

ALGORITHM 14.33 Kaltofen & Lobo's algorithm.
Input: A monic squarefree polynomial $f \in \mathbb{F}_q[x]$ of degree $n > 0$.
Output: If f is reducible, then a proper factor of f, otherwise f itself, or "failure".

{ $Q \in \mathbb{F}_q^{n\times n}$ denotes the Petr-Berlekamp matrix of f }

1. randomly choose two column vectors $u, v \in \mathbb{F}_q^n$
   call the iterated Frobenius algorithm 14.26 to compute $Q^i v$ for $0 \leq i < 2n$
   call Algorithm 12.9 with input $u^T Q^i v$ for $0 \leq i < 2n$ to compute the minimal polynomial $m \in \mathbb{F}_q[x]$ of the sequence $(u^T Q^i v)_{i\geq 0} \in \mathbb{F}_q^{\mathbb{N}}$
   { with "high" probability, m is the minimal polynomial of Q; see Exercise 12.15 }
2. **if** $m = x^n - 1$ **return** f **else if** $m(1) \neq 0$ **return** "failure"
3. $h \longleftarrow \dfrac{m(x+1)}{x}$
   choose another random vector $w \in \mathbb{F}_q^n$
   call the iterated Frobenius algorithm 14.26 to compute $Q^i w$ for $0 \leq i \leq \deg h$
   compute the polynomial $a \in \mathbb{F}_q[x]$ with coefficient vector $h(Q) \cdot w$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

4. $g_1 \longleftarrow \gcd(a, f)$
   if $g_1 \neq 1$ and $g_1 \neq f$ **then return** g_1
5. **call** the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $b = a^{(q-1)/2} \text{ rem } f$
6. $g_2 \longleftarrow \gcd(b-1, f)$
   if $g_2 \neq 1$ and $g_2 \neq f$ **then return** g_2 **else return** "failure"

The analyses in Kaltofen & Lobo (1994) and Kaltofen & Shoup (1998) imply the following.

THEOREM 14.34.
Algorithm 14.33 works correctly as specified, returns "failure" with probability at most $1/2$ if $q \geq 4n$, and takes $O(\mathsf{M}(n^2)\log n + \mathsf{M}(n)\log q)$ operations in $\mathbb{F}_q$. If the algorithm is used recursively to factor f completely, then the expected recursion depth is $O(\log_p n \cdot \log r)$, where $p = \operatorname{char}\mathbb{F}_q$ and r is the number of irreducible factors of f.

Figure 14.9 illustrates how the asymptotic running times of four factorization algorithms depend on the relation between the two independent parameters n, the degree of the input polynomial, and $\log_2 q$. The unit of time is $\log_2 q$ bit operations, which is the cost of one operation in $\mathbb{F}_q$. We ignore factors $n^{o(1)}$. The figure, based on a similar one in Kaltofen & Shoup (1998), abstracts a three-dimensional picture of the running time as a function of n and $\log_2 q$ into a two-dimensional figure with two logarithmic axes x and y, where $\log_2 q$ and the time are about n^x and n^y, respectively. The figure pictures Berlekamp's classical algorithm 14.31, the method of Cantor & Zassenhaus (Algorithms 14.3 and 14.10), the iterated Frobenius algorithm of von zur Gathen & Shoup (Corollary 14.30), and the subquadratic algorithm of Kaltofen & Shoup (1998), incorporating Huang & Pan's (1998) fast rectangular matrix multiplication. A derivation of the (rounded) numerical value in Figure 14.9 for the latter is given in Notes 14.8. Huang & Pan (1998) present another algorithm whose running time, corresponding to $x + 1.80535$, beats the others for $x \leq 0.00173$; its graph is virtually indistinguishable for these small values from the lower left segment. Finally, Kedlaya & Umans (2009) have applied their fast modular composition algorithm, mentioned at the end of Section 12.2, to obtain a factoring algorithm using an expected number of $O(n^{1.5+o(1)}\log q + n\log^2 q)$ of bit operations. They achieve this by moving out of the algebraic model of using only operations in $\mathbb{F}_q$. Their operations count amounts to about $n^{1.5} + n\log q$ $\mathbb{F}_q$-equivalent operations. Each of these six algorithms is asymptotically faster than previously known methods for some choices of n and q.

Computationally, (fast) polynomial factorization over a finite field is a much more advanced task than, for example, multiplication or even gcd computation.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

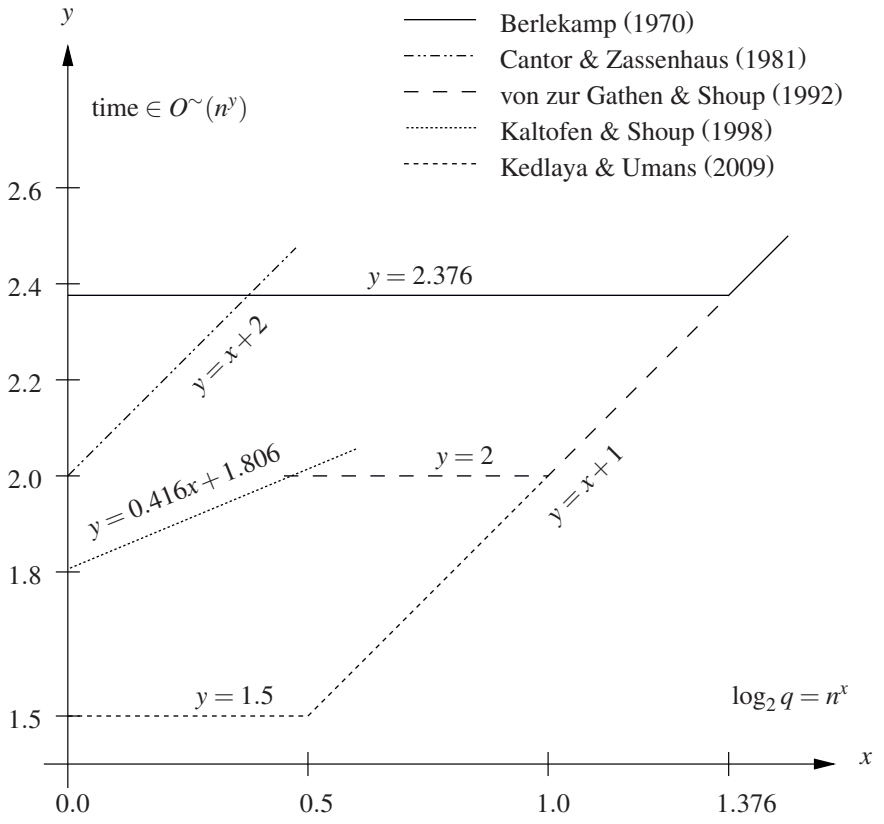


FIGURE 14.9: Asymptotic running times of various factoring algorithms.

Before implementing a particular algorithm, one has to implement carefully many other routines for basic polynomial arithmetic, and designing a polynomial factorization package may require several woman-years. Figure 14.10 gives an overview of various factorization algorithms for univariate polynomials over finite fields and the polynomial arithmetic they are based on. The arrows indicate dependencies.

14.9. Testing irreducibility and constructing irreducible polynomials

Any factorization algorithm can be used as an irreducibility test. For example, in the distinct-degree factorization algorithm 14.3 we can stop when we either find a proper factor or reach degree more than $n/2$ without having found a factor. An alternative is provided by the following corollary.

COROLLARY 14.35.
A polynomial $f \in \mathbb{F}_q[x]$ of degree $n \geq 1$ is irreducible if and only if

(i) f divides $x^{q^n} - x$, and

(ii) $\gcd(x^{q^{n/t}} - x, f) = 1$ for all prime divisors t of n.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

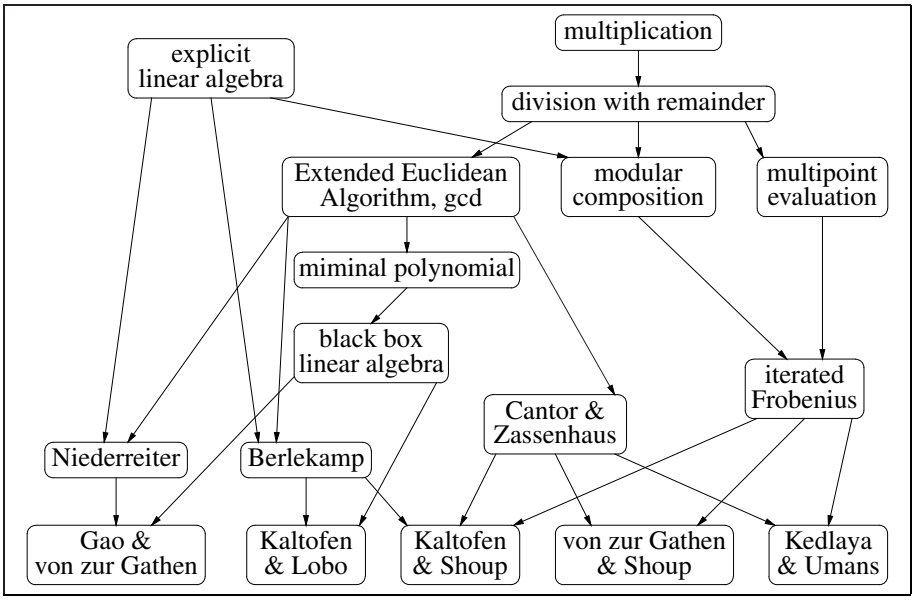


FIGURE 14.10: Algorithms for polynomial factorization over finite fields.

PROOF. It follows immediately from Theorem 14.2 that f satisfies the two conditions if it is irreducible. Conversely, if (i) holds, then Theorem 14.2 implies that the degree of any irreducible factor of f divides n. Let g be such an irreducible factor, and suppose that $d = \deg g < n$. Then d divides n/t for some prime factor t of n, and hence $g \mid x^{q^{n/t}} - x$. This contradicts (ii), and we conclude that $d = n$ and f is irreducible. □

ALGORITHM 14.36 Irreducibility test over finite fields.
Input: $f \in \mathbb{F}_q[x]$ of degree n.
Output: "irreducible" or "reducible".

1. **call** the repeated squaring algorithm 4.8 to compute x^q rem f
   use the modular composition algorithm 12.3 to compute $a = x^{q^n}$ rem f
   if $a \neq x$ **then return** "reducible"

2. **for** all prime divisors t of n **do**

3. use the modular composition algorithm 12.3 to compute $b = x^{q^{n/t}}$ rem f
   if $\gcd(b - x, f) \neq 1$ **then return** "reducible"

4. **return** "irreducible"

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

For an integer $n \geq 1$, we denote by $\delta(n)$ the number of distinct prime divisors of n. Since 2 is the smallest prime, we have the trivial upper bound $\delta(n) \leq \log_2 n$. In fact, $\delta(n) \leq \ln n / \ln\ln n$, and $\delta(n)$ is $O(\log\log n)$ on average.

THEOREM 14.37.
Algorithm 14.36 correctly decides whether the input polynomial is irreducible. It can be implemented so as to use $O(\mathsf{M}(n)\log q + (n^{(\omega+1)/2} + n^{1/2}\mathsf{M}(n))\delta(n)\log n)$ *or* $O^{\sim}(n^{(\omega+1)/2} + n\log q)$ *operations in* $\mathbb{F}_q$.

PROOF. Correctness follows from Corollary 14.35. The cost for computing x^q rem f in step 1 is $O(\mathsf{M}(n)\log q)$ field operations. To compute $s_m = x^{q^m}$ rem f for some $m \in \mathbb{N}$, we employ the polynomial representation (6) of the Frobenius map, noting that

$$x^{q^{i+j}} \bmod f = \left(\xi^{q^i}\right)^{q^j} = s_i(\xi)^{q^j} = s_i(\xi^{q^j}) = s_i(s_j(\xi)) = s_i(s_j) \bmod f$$

for all i, j. Thus x^{q^m} rem f can be computed from x^q rem f in a "repeated squaring" fashion along the binary representation of m, taking $O(\log m)$ modular composition steps of the form $s_i(s_j)$ rem f. By Theorem 12.4, this can be done at a total cost of $O((n^{(\omega+1)/2} + n^{1/2}\mathsf{M}(n))\log m)$ operations in $\mathbb{F}_q$, dominating the cost for the gcd in step 3. The total number of times we have to compute some s_m is $1 + \delta(n)$, and the claim follows since $m \leq n$ in all those cases. □

With the current world record $\omega < 2.373$ (Section 12.1), we have $(\omega+1)/2 < 1.687$. The iterated Frobenius algorithm for distinct-degree factorization can be used for testing irreducibility and takes $O^{\sim}(n^2 + n\log q)$ operations in $\mathbb{F}_q$ (Corollary 14.28). A third irreducibility test (for a squarefree polynomial) is given by (9); it is sufficient to compute the rank of $Q - I$, taking $O(n^{\omega} + \mathsf{M}(n)\log q)$ field operations.

Comparing the three tests and using classical matrix arithmetic, where $\omega = 3$, the first two give the same soft-Oh estimate $n^2 + n\log q$, but the Oh-bound shows that the test 14.36 is faster: $n^2\delta(n)\log n$ versus $\mathsf{M}(n^2)\log n$, for small q. The n^3 estimate for the third method is in a different league. When we take $\omega < 3$, say $\omega = 2.373$ (Section 12.1), then the estimate for Algorithm 14.36 shrinks to only $O^{\sim}(n^{1.687} + n\log q)$. The run time of a method by Kedlaya & Umans (2009) corresponds, as explained at the end of the previous section, to only $n^{1.5+o(1)} + O^{\sim}(n\log q)$ operations in $\mathbb{F}_q$.

Now that we know how to test a polynomial for irreducibility, it is natural to ask how to find irreducible polynomials. This is used to construct finite extension fields of finite fields and in modular algorithms. The following result tells us how frequently irreducible polynomials occur among arbitrary polynomials.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

LEMMA 14.38. *Let q be a prime power and $n \in \mathbb{N}_{\geq 1}$. Then the number $I(n,q)$ of monic irreducible polynomials of degree n in $\mathbb{F}_q[x]$ satisfies*

$$\frac{q^n - 2q^{n/2}}{n} \leq I(n,q) \leq \frac{q^n}{n}.$$

In particular, if $q^n \geq 16$, then the probability p_n for a uniformly random monic polynomial of degree n to be irreducible satisfies

$$\frac{1}{2n} \leq \frac{1}{n}\left(1 - \frac{2}{q^{n/2}}\right) \leq p_n \leq \frac{1}{n}.$$

PROOF. Let f_n be the product of all monic irreducible polynomials of degree n in $\mathbb{F}_q[x]$. Thus $\deg f_n = n \cdot I(n,q)$. Theorem 14.2 can then be reformulated as

$$x^{q^n} - x = \prod_{d|n} f_d = f_n \cdot \prod_{d|n, d<n} f_d.$$

Taking degrees on both sides, we obtain $q^n = \deg f_n + \sum_{d|n, d<n} \deg f_d$, and hence

$$q^n \geq \deg f_n = n \cdot I(n,q). \tag{10}$$

This proves the upper bound. Now

$$\sum_{d|n, d<n} \deg f_d \leq \sum_{1 \leq d \leq n/2} \deg f_d \leq \sum_{1 \leq d \leq n/2} q^d < \frac{q^{n/2+1} - 1}{q-1} \leq 2q^{n/2},$$

by (10) with d instead of n and the fact that $q \geq 2$, and hence

$$n \cdot I(n,q) = \deg f_n = q^n - \sum_{d|n, d<n} \deg f_d \geq q^n - 2q^{n/2},$$

which establishes the lower bound.

There are q^n monic polynomials of degree n in $\mathbb{F}_q[x]$, and hence

$$\frac{1}{n} \geq \frac{I(n,q)}{q^n} \geq \frac{1}{n}(1 - 2q^{-n/2}) \geq \frac{1}{2n}$$

if $q^n \geq 16$. This proves the last assertion. □

In fact, the probability is close to $1/n$ when q^n is not too small. The precise formula

$$n \cdot I(n,q) = \sum_{d|n} \mu\left(\frac{n}{d}\right) q^d$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

can be found by using a well-known number theoretic tool called **Möbius inversion** (Exercise 14.46). Here

$$\mu(n) = \begin{cases} 1 & \text{if } n = 1, \\ (-1)^k & \text{if } n \text{ is the product of } k \text{ distinct primes}, \\ 0 & \text{if } n \text{ is not squarefree}, \end{cases} \tag{11}$$

is the **Möbius function**, defined for positive integers n. The first few values of μ are listed in Section 17.4. Table 14.11 tabulates the values of $I(n,q)$ for some small values of n and q.

n	$q=2$	$q=3$	$q=4$	$q=5$	$q=7$	$q=8$	$q=9$
2	1	3	6	10	21	28	36
3	2	8	20	40	112	168	240
4	3	18	60	150	588	1008	1620
5	6	48	204	624	3360	6552	11 808
6	9	116	670	2580	19 544	43 596	88 440
7	18	312	2340	11 160	117 648	299 592	683 280
8	30	810	8160	48 750	720 300	2 096 640	5 380 020
9	56	2184	29 120	217 000	4 483 696	14 913 024	43 046 640
10	99	5880	104 754	976 248	28 245 840	107 370 900	348 672 528

TABLE 14.11: The number $I(n,q)$ of irreducible polynomials for $2 \le n \le 10$ and $q \le 9$.

A simple idea to find a random irreducible polynomial of a given degree n is now to repeatedly and independently choose random polynomials of degree n and test them for irreducibility. Using Algorithm 14.36, this leads to the following result.

COROLLARY 14.39.
For a prime power q and $n \in \mathbb{N}_{>0}$, one can find a uniformly random irreducible polynomial of degree n in $\mathbb{F}_q[x]$ using an expected number of

$$O(n\mathsf{M}(n)\log q + (n^{(\omega+3)/2} + n^{3/2}\mathsf{M}(n))\delta(n)\log n) \text{ or } O^{\sim}(n^{(\omega+3)/2} + n^2 \log q)$$

operations in $\mathbb{F}_q$.

The exponent $(\omega+3)/2$ is less than 2.688 for the smallest currently known ω. The following alternative method is somewhat faster.

ALGORITHM 14.40 Ben-Or's generation of irreducible polynomials.
Input: A prime power q and $n \in \mathbb{N}_{>0}$.
Output: A uniformly random monic irreducible polynomial of degree n in $\mathbb{F}_q[x]$.

1. randomly choose a monic polynomial $f \in \mathbb{F}_q[x]$ of degree n

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

2. **for** $i = 1, \ldots, \lfloor n/2 \rfloor$ **do**
   $g_i \longleftarrow \gcd(x^{q^i} - x, f)$, **if** $g_i \neq 1$ **then goto** 1
3. **return** f

By Theorem 14.4, step 2 can be performed with $O^{\sim}(n^2 \log q)$ field operations, and Lemma 14.38 would imply a total cost of $O^{\sim}(n^3 \log q)$, but the following analysis shows that the actual cost is lower by about one order of magnitude. We state the following property without proof.

LEMMA 14.41. *Let q be a prime power and $n \in \mathbb{N}_{>0}$. The expected value of the degree of the smallest irreducible factor of a uniformly random polynomial of degree n in $\mathbb{F}_q[x]$ is $O(\log n)$.*

THEOREM 14.42.
Ben-Or's algorithm 14.40 works correctly as specified and takes an expected number of $O(n\mathsf{M}(n)\log n \log(nq))$ or $O^{\sim}(n^2 \log q)$ operations in $\mathbb{F}_q$.

PROOF. A reducible polynomial has an irreducible factor of degree at most $n/2$, which divides some g_i, by Theorem 14.2. This proves the correctness.

The test for a single value of i takes $O(\mathsf{M}(n)\log(nq))$ operations in $\mathbb{F}_q$, and the expected number of operations for step 2 for a single f is $O(\mathsf{M}(n)\log n \log(nq))$, by Lemma 14.41. The expected number of trials is $O(n)$, by Lemma 14.38. □

In practice, it may be advantageous to use a hybrid algorithm which switches from the distinct-degree factorization in step 2 of Algorithm 14.40 to Algorithm 14.36 as soon as the former has certified that the polynomial in question has no irreducible factors below a certain bound, say $\log_2 n$. A random polynomial of degree n without factors of degree up to $m \in O(\log n)$ is irreducible with probability about cm/n, for some constant c. Experiments for $q = 2$ in Section 15.7, where variants of the distinct-degree factorization algorithm and of Algorithm 14.36 were run in parallel, indicate that this is useful even when classical matrix multiplication with $\omega = 3$ is used.

COROLLARY 14.43.
For a prime power q and $n \in \mathbb{N}$, we can construct the extension field $\mathbb{F}_{q^n}$ of $\mathbb{F}_q$ using an expected number of $O(n\mathsf{M}(n)\log n \log(nq))$ or $O^{\sim}(n^2 \log q)$ operations in $\mathbb{F}_q$.

For the big prime modular gcd algorithm 6.28 in $\mathbb{F}_q[x, y]$, we have to find an irreducible polynomial which does not divide some unknown resultant $r \in \mathbb{F}_q[y]$, on which only a degree bound $\deg r \leq m$ is known.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

COROLLARY 14.44.
Let $n \in \mathbb{N}$, q be a prime power, and $r \in \mathbb{F}_q[y]$ nonzero of degree at most m. Then we can compute a uniformly random irreducible polynomial $f \in \mathbb{F}_q[y]$ of degree n taking $O(n\mathsf{M}(n)\log n \log(nq))$ or $O^{\sim}(n^2 \log q)$ operations in $\mathbb{F}_q$, and f does not divide r with probability at least $1/2$ if $q^n \geq 2m$.

PROOF. The cost estimate is from Theorem 14.42. There are $I(n,q)$ irreducible polynomials of degree n, and at most $\lfloor m/n \rfloor$ of them divide r. Thus the probability that f does not divide r is

$$\frac{I(n,q) - \left\lfloor \frac{m}{n} \right\rfloor}{I(n,q)} \geq 1 - \frac{m}{n} \cdot \frac{n}{q^n} = 1 - \frac{m}{q^n},$$

and the claim follows. □

In fact, the probability is close to 1 when q^n is not too small.

14.10. Cyclotomic polynomials and constructing BCH codes

We recall from Section 8.2 that an element ω in a field F is an **nth root of unity** if $\omega^n = 1$, and a **primitive nth root of unity** if in addition $\operatorname{char} F \nmid n$ and $\omega^k \neq 1$ for $1 \leq k < n$, or equivalently, ω has multiplicative order n in $F^\times$.

DEFINITION 14.45. *The polynomial*

$$\Phi_n = \prod_{\substack{\omega \in \mathbb{C} \text{ primitive} \\ n\text{th root of unity}}} (x - \omega) = \prod_{\substack{1 \leq k < n \\ \gcd(k,n)=1}} (x - e^{2\pi i k/n}) \in \mathbb{C}[x]$$

*is called the **nth cyclotomic polynomial**.*

Lemma 14.47 below implies that Φ_n has coefficients in $\mathbb{Z}$. Table 14.12 lists the first 20 cyclotomic polynomials. We have $\deg \Phi_n = \varphi(n)$, where φ is Euler's totient function (Section 4.2).

LEMMA 14.46. *For a positive integer n, we have the factorization*

$$x^n - 1 = \prod_{d \mid n} \Phi_d. \tag{12}$$

PROOF. Let $\omega \in \mathbb{C}$ be a zero of $x^n - 1$, that is, an nth root of unity. Then $\operatorname{ord}(\omega) = d$ for some divisor d of n, by Lagrange's theorem (Section 25.1). But this means that

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

n	Φ_n	n	Φ_n
1	$x-1$	11	$x^{10}+x^9+\cdots+x+1$
2	$x+1$	12	x^4-x^2+1
3	x^2+x+1	13	$x^{12}+x^{11}+\cdots+x+1$
4	x^2+1	14	$x^6-x^5+x^4-x^3+x^2-x+1$
5	$x^4+x^3+x^2+x+1$	15	$x^8-x^7+x^5-x^4+x^3-x+1$
6	x^2-x+1	16	x^8+1
7	$x^6+x^5+x^4+x^3+x^2+x+1$	17	$x^{16}+x^{15}+\cdots+x+1$
8	x^4+1	18	x^6-x^3+1
9	x^6+x^3+1	19	$x^{18}+x^{17}+\cdots+x+1$
10	$x^4-x^3+x^2-x+1$	20	$x^8-x^6+x^4-x^2+1$

TABLE 14.12: The first 20 cyclotomic polynomials.

ω is a primitive dth root of unity, and hence $\Phi_d(\omega)=0$. Conversely, if $\omega\in\mathbb{C}$ is a primitive dth root of unity for some divisor d of n, then $\omega^d=1$, and hence also $\omega^n=1$. This shows that both polynomials in (12) have the same roots in $\mathbb{C}$.

Now $(x^n-1)'=nx^{n-1}$ is coprime to x^n-1, which implies that x^n-1 is squarefree. By definition, Φ_d is squarefree for all d, and Φ_d and Φ_e have no roots in common unless $d=e$. This proves that also the polynomial on the right hand side is squarefree, and since both polynomials are monic, they are equal. □

As examples, we have

$$x^6-1=(x^2-x+1)(x^2+x+1)(x+1)(x-1)=\Phi_6\Phi_3\Phi_2\Phi_1,$$
$$x^8-1=(x^4+1)(x^2+1)(x+1)(x-1)=\Phi_8\Phi_4\Phi_2\Phi_1.$$

Using Möbius inversion, Lemma 14.46 yields the formula

$$\Phi_n=\prod_{d|n}(x^d-1)^{\mu(n/d)}$$

for the cyclotomic polynomials. For example, we have

$$\Phi_6=x^2-x+1=\frac{(x^6-1)(x-1)}{(x^3-1)(x^2-1)}.$$

The following lemma, which is proven in Exercise 14.45, yields an alternative method for computing Φ_n.

LEMMA 14.47. *Let $n,k\in\mathbb{N}_{>0}$. Then*

(i) $\Phi_n=x^{n-1}+x^{n-2}+\cdots+x+1$ if n is prime,

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(ii) $\Phi_{2n} = \Phi_n(-x)$ *if* $n \geq 3$ *is odd,*

(iii) $\Phi_{kn}\Phi_n = \Phi_n(x^k)$ *if* k *is a prime not dividing* n*,*

(iv) $\Phi_{kn} = \Phi_n(x^k)$ *if every prime divisor of* k *divides* n*.*

ALGORITHM 14.48 Cyclotomic polynomial computation.
Input: $n \in \mathbb{N}_{>0}$ and the distinct prime divisors $p_1, \ldots, p_r$ of n.
Output: $\Phi_n \in \mathbb{Z}[x]$.

1. $f_0 \longleftarrow x - 1$
2. **for** $i = 1, \ldots, r$ **do** $f_i \longleftarrow \dfrac{f_{i-1}(x^{p_i})}{f_{i-1}}$
3. **return** $f_r(x^{n/(p_1 \cdots p_r)})$

THEOREM 14.49.
Algorithm 14.48 uses $O(\mathsf{M}(n)\log n)$ *arithmetic operations in* $\mathbb{Z}$ *and correctly computes the* n*th cyclotomic polynomial.*

PROOF. We claim that $f_i = \Phi_{p_1 \cdots p_i}$ for $0 \leq i \leq r$. This is clear if $i = 0$ and follows inductively from Lemma 14.47 (iii). Now let $m = p_1 \cdots p_r$ be the squarefree part of n. Then

$$f_r(x^{n/m}) = \Phi_m(x^{n/m}) = \Phi_n,$$

by Lemma 14.47 (iv), since every prime divisor of n/m divides m.

The only arithmetic operations occur in step 2, where the ith operation is a division of a polynomial of degree $(p_1 - 1)\cdots(p_{i-1} - 1)p_i$ by a polynomial of degree $(p_1 - 1)\cdots(p_{i-1} - 1)$. Roughly, this takes $O(\mathsf{M}(n))$ additions and multiplications in $\mathbb{Z}$ (since the divisor is monic), and the claim follows since $r \leq \log_2 n$. □

Now let F be an arbitrary field and $n \in \mathbb{N}$. We may consider Φ_n as a polynomial in $F[x]$ by reducing its coefficients modulo the characteristic p of F. Then (12), Lemma 14.47, and Algorithm 14.48 are still valid. In Galois theory, it is shown that if p does not divide n, then

$$\Phi_n = \prod_{\substack{\omega \in E \text{ primitive} \\ n\text{th root of unity}}} (x - \omega)$$

holds over any extension E of F containing a primitive nth root of unity, and that Φ_n is irreducible over $\mathbb{Q}$ (so that (12) is the irreducible factorization of $x^n - 1$ over $\mathbb{Q}$). The following lemma says that the latter is not true over finite fields.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

LEMMA 14.50. *Let $n \in \mathbb{N}_{>0}$, $\mathbb{F}_q$ be a finite field of characteristic p not dividing n, and $d = \mathrm{ord}_n(q)$ the multiplicative order of q in $\mathbb{Z}_n^\times$. Then Φ_n factors in $\mathbb{F}_q[x]$ into a product of $\varphi(n)/d$ irreducible polynomials of degree d each. In particular, the degree of the minimal polynomial of any primitive nth root of unity over $\mathbb{F}_q$ is d.*

PROOF. First we note that $d \mid \varphi(n) = \#\mathbb{Z}_n^\times$, by Lagrange's theorem. Now n divides $q^d - 1 = \#\mathbb{F}_{q^d}^\times$, and hence $\mathbb{F}_{q^d}$ contains a primitive nth root of unity ω (Lemma 8.8). We choose such an ω, and let $f \in \mathbb{F}_q[x]$ be the unique irreducible factor of Φ_n that has ω as a root. Since $f(x^q) = f(x)^q$, the element ω^{q^i} is a root of f for all $i \in \mathbb{N}$. Now $1, q, q^2, \ldots, q^{d-1}$ are distinct modulo n, the order of ω in $\mathbb{F}_{q^d}^\times$, and hence $\{\omega, \omega^q, \omega^{q^2}, \ldots, \omega^{q^{d-1}}\}$ are d distinct roots of f. Thus $\deg f \geq d$. On the other hand, we have $\mathbb{F}_q[x]/\langle f \rangle \cong \mathbb{F}_q(\omega) \subseteq \mathbb{F}_{q^d}$ (Section 25.3), whence $\deg f \leq d$. Thus $\deg f = d$, and since the choice of ω was arbitrary, this is true for all irreducible factors of Φ_n. □

For example, the order of 3 modulo 8 is 2, and in fact Φ_8 splits into two irreducible factors of degree 2 over $\mathbb{F}_3$: $x^4 + 1 = (x^2 + x - 1)(x^2 - x - 1)$.

EXAMPLE 14.51. We take $q = 2$ and $n = 15$. Then $d = \mathrm{ord}_{15}(2) = 4$. The polynomial $x^{15} - 1$ factors in $\mathbb{F}_2[x]$ as

$$\begin{aligned} x^{15} - 1 &= \Phi_{15}\Phi_5\Phi_3\Phi_1 \\ &= (x^8 - x^7 + x^5 - x^4 + x^3 - x + 1)(x^4 + x^3 + x^2 + x + 1)(x^2 + x + 1)(x - 1) \\ &= (x^4 + x + 1)(x^4 + x^3 + 1)(x^4 + x^3 + x^2 + x + 1)(x^2 + x + 1)(x + 1). \end{aligned}$$

As predicted by Lemma 14.50, Φ_{15} splits into two irreducible factors of degree 4, and Φ_5, Φ_3, and Φ_1 remain irreducible. Let $\beta \in \mathbb{F}_{16}$ be a root of $x^4 + x + 1$. Then β is a primitive 15th root of unity. The roots of the minimal polynomial $x^4 + x + 1$ of β are $\beta, \beta^2, \beta^4, \beta^8$.

For $i \in \mathbb{Z}$, β^i is a primitive lth root of unity, where $l = \mathrm{ord}(\beta^i) = n/\gcd(n, i)$ (Exercise 8.13). We have $\mathrm{ord}(\beta^3) = 15/\gcd(3, 15) = 5$, so that β^3 is a primitive 5th root of unity. Now $\mathrm{ord}_5(2) = 4$, so that the minimal polynomial of β^3 has precisely the four roots $\beta^3, \beta^6, \beta^{12}, \beta^{24} = \beta^9$. Similarly, $\mathrm{ord}(\beta^5) = 3$ and $\mathrm{ord}_3(2) = 2$, whence β^5 is a primitive third root of unity and its minimal polynomial only has the two roots β^5, β^{10}. ◇

We define an equivalence relation $\sim$ on $\mathbb{Z}_n$ by

$$i \sim j \iff \exists l \in \mathbb{Z} : iq^l = j. \tag{13}$$

If $i \in \mathbb{Z}_n^\times$, then the equivalence class of i is the **cyclotomic coset** $i \cdot \langle q \rangle$ of the cyclic subgroup $\langle q \rangle$ of $\mathbb{Z}_n^\times$. If $d = \mathrm{ord}_n(q)$ and $\beta \in \mathbb{F}_{q^d}$ is a primitive nth root of unity,

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

then the powers β^i and β^j have the same minimal polynomial if and only if $i \sim j$, as in the example.

Lemma 14.50 implies that cyclotomic polynomials can be directly factored over finite fields using equal-degree factorization, without performing either squarefree or distinct-degree factorization. The cost is $O^\sim(n^2 + n\log q)$ operations in $\mathbb{F}_q$ or $O^\sim(n^2 \log q + n\log^2 q)$ word operations if $p \nmid n$. Exercise 14.47 yields an even faster algorithm for factoring $x^n - 1$, taking only $O^\sim(n\log^2 q)$ word operations, which can be modified so as to work for Φ_n as well. When p divides n, we have $\Phi_n = \Phi_{n/p}^p$ in $\mathbb{F}_q[x]$.

In Chapter 7, we discussed a class of cyclic codes that are of importance in modern coding theory, the **BCH codes**. For a finite field $\mathbb{F}_q$, a primitive nth root of unity $\beta \in \mathbb{F}_{q^d}$ in some extension field of $\mathbb{F}_q$, and a positive integer δ, BCH(q,n,δ) is the cyclic code (that is, the ideal) in $\mathbb{F}_q[x]/\langle x^n - 1\rangle$ generated by $g \bmod x^n - 1$, where $g \in \mathbb{F}_q[x]$ is the least common multiple of the minimal polynomials of $\beta, \beta^2, \ldots, \beta^{\delta-1}$ over $\mathbb{F}_q$. We now show how to compute such a generator polynomial g.

ALGORITHM 14.52 Construction of BCH codes.
Input: A prime power q and positive integers $n \geq \delta$ with $\gcd(n,q) = 1$.
Output: A generator polynomial $g \in \mathbb{F}_q[x]$ of a BCH(q,n,δ) code.

1. let $p_1, \ldots, p_r \in \mathbb{N}$ be the distinct prime divisors of n
   call Algorithm 14.48 over $\mathbb{F}_q$ to compute Φ_n
2. use equal-degree factorization to find one irreducible factor $f \in \mathbb{F}_q[x]$ of Φ_n
   $\beta \longleftarrow x \bmod f$
3. determine the distinct $\sim$ equivalence classes $S_1, \ldots, S_t \subseteq \mathbb{Z}_n$ of $1, \ldots, \delta - 1$
4. compute $\beta^2, \beta^3, \ldots, \beta^{\delta-1}$
5. **for** $k = 1, \ldots, t$ **do**
   $i \longleftarrow \min S_k, \quad m \longleftarrow \#S_k$
   compute $\beta^{2i}, \ldots, \beta^{(2m-1)i}$
   use Exercise 12.10 to compute the minimal polynomial $g_k \in \mathbb{F}_q[x]$ of β^i
6. **return** $g_1 \cdots g_t$

Before analyzing the algorithm, we give an example. Let $1 \leq i < \delta$ and $l = \text{ord}(\beta^i)$. By Lemma 14.50, the degree of the minimal polynomial of β^i over $\mathbb{F}_q$ is $m = \text{ord}_l(q)$ and its roots are precisely $\beta^i, \beta^{iq}, \ldots, \beta^{iq^{m-1}}$, and $\{i, iq, \ldots, iq^{m-1}\}$ is the $\sim$ equivalence class of i in step 3.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

EXAMPLE 14.51 (continued). We take $\delta = 7$, corresponding to the fourth row in Table 7.1. The distinct $\sim$ equivalence classes of $1,\ldots,6$ are

$$S_1 = \{1,2,4,8\}, \quad S_2 = \{3,6,9,12\}, \quad S_3 = \{5,10\}.$$

Suppose that we have $f = x^4 + x + 1$ in step 2. Then $g_1 = f$ in step 5, $g_2 = \Phi_5$ ($\beta^3, \beta^6, \beta^9, \beta^{12}$ are precisely the primitive 5th roots of unity), and $g_3 = \Phi_3$ (β^5, β^{10} are the primitive 3rd roots of unity). Thus

$$g = g_1 g_2 g_3 = x^{10} + x^8 + x^5 + x^4 + x^2 + x + 1 \in \mathbb{F}_2[x]$$

generates a BCH$(2,15,6)$ code. Its minimal distance is 7. $\diamond$

THEOREM 14.53.
Algorithm 14.52 works correctly as specified and takes $O(\mathsf{M}(nd)(n/d)\log n + \mathsf{M}(n)\log q)$ *or* $O^{\sim}(n^2 + n\log q)$ *operations in* $\mathbb{F}_q$, *where* $d = \mathrm{ord}_n(q)$.

PROOF. Since β is a root of Φ_n, it is a primitive nth root of unity. For $1 \le j < \delta$, the minimal polynomial of β^j is equal to g_k for the unique $k \in \{1,\ldots,t\}$ such that $j \in S_k$, and hence the least common multiple of the minimal polynomials of all β^j for $1 \le j < \delta$ is $g_1 \cdots g_t$. This proves correctness.

The cost for step 1 is $O(\mathsf{M}(n)\log n)$ operations in $\mathbb{F}_q$, by Theorem 14.49. Step 2 can be done with $O(\mathsf{M}(nd)(n/d)\log n + \mathsf{M}(n)\log q)$ operations, by the discussion following Corollary 14.29. All powers of β needed in steps 4 and 5 may be computed with at most $\min\{\delta - 2 + t(2d-2), n-2\}$ multiplications in $\mathbb{F}_{q^d}$, or $O(\min\{\delta d, n\}\mathsf{M}(d))$ operations in $\mathbb{F}_q$, since $t \le \delta - 1$. The cost for one minimal polynomial computation in step 5 is $O(\mathsf{M}(m)\log m)$ operations in $\mathbb{F}_q$, by Exercise 12.10. The total cost for steps 4 and 5 is $O(\min\{\delta d, n\}\mathsf{M}(d) + \delta\mathsf{M}(d)\log d)$, since $m \le d$ in step 5. Finally, multiplying the g_k together may be done with $O(\mathsf{M}(n)\log t)$ operations, by Lemma 10.4. The total cost is dominated by the cost for step 2 since $d \le \varphi(n) < n$ and $\delta \le n$. $\square$

Notes. The pioneering works for this area of computer algebra are those of Berlekamp (1967, 1970), Zassenhaus (1969), and Cantor & Zassenhaus (1981).

14.2 and 14.3. The distinct-degree factorization of Section 14.2 appears in Zassenhaus (1969), Kempfert (1969), Knuth (1998), already in the 1969 edition, Berlekamp (1970), and Cantor & Zassenhaus (1981); the latter also contains the equal-degree factorization of Section 14.3.

Actually the basic algorithms go back almost two centuries. Gauß' *Disquisitiones Generales de Congruentiis* were to appear as Part 8 of his *Disquisitiones Arithmeticae*, but did not make it (see page 372). Written in 1797 or 1798, but not quite polished to the Master's usual high gloss, his hand-written notes were published in his Nachlass (Gauß 1863a, 1863b). In article 370, Gauß writes: *Sit itaque X functio, quae nullos amplius divisores*

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

aequales involvit. Supra vidimus, $x^p - x$ esse productum ex omnibus functionibus primis unius dimensionis. Sit ξ divisor communis maximae dimensionis functionum X et $x^p - x$, erit ξ productum ex omnibus divisoribus ipsius X unius dimensionis et $\frac{X}{\xi}$ huiusmodi divisores non amplius habebit. Quodsi autem inveniatur, functiones X et $x^p - x$ esse inter se primas, X nullum divisorem unius dimensionis habebit adeoque congruentia $X \equiv 0$ radices reales non habebit. Porro quoniam $x^{pp} - x$ est productum ex omnibus functionibus primis duarum dimensionum uniusque, divisor communis maximae dimensionis functionum $x^{pp} - x$ et $\frac{X}{\xi}, \xi'$ involvet omnes divisores ipsius X, qui sunt duarum dimensionum. Hinc ulterius progrediendo perspicitur, X hoc modo in factores ξ, ξ', ξ'' etc. resolvi, qui continent respective omnes divisores unius, duarum, trium etc. dimensionum[1].

The algorithm was found independently and first published by Galois (1830) (who forgot to say that the gcd has to be removed before the next step and the possibility that f' might be zero). Serret's (1866) book contains a correct version of Galois's algorithm. Arwin's (1918) paper contains many of the modern ideas on factorization, including this algorithm. Since Cantor & Zassenhaus (1981) it is a staple of modern computer algebra.

Theorem 14.2 is in Gauß (1863b), article 353, for a prime q. Galois (1830) also discovered and was the first to publish it. In his article 372, Gauß begins with the simplest case of equal-degree factorization, where all factors are linear. He points out the analogy with factoring integers, but ends on a somewhat helpless note: *Sed huic rei inhaerere nolumus, nam calculator exercitatus principia probe assecutus, quando opus est, facile artificia particularia reperiet*[2].

Legendre (1785) already knew the basics of the probabilistic root finding method, via Algorithm 14.10. He suggests (§§25–28) splitting f via $\gcd(f, x^{(A-1)/2} \pm 1)$, where A is a prime, and to translate the variable x if necessary: *On cherchera* [...] *la valeur de* $x^{\frac{A-1}{2}}$ *exprimée en puissances de x inférieures à* x^n, *& on égalera cette valeur à* $+1$ *&* -1 *successivement.* [...] *Toutes les fois que l'équation proposée aura des racines de deux espèces, les unes au nombre de p, donnant* $x^{\frac{A-1}{2}} = 1$, *les autres au nombre de q, donnant* $x^{\frac{A-1}{2}} = -1$; *la séparation en sera faite par la méthode précédente.* [...] *On peut faire* $x = y \pm k, k$ *étant à volonté, & résoudre l'équation en y par les mêmes principes.*[3] Euler (1761) proved Lemma 14.6 (ii), and Euler (1754/55) showed Lemma 14.7 (i), both in the case where q is prime. He introduced the terminology of *residuum* and *non-residuum*; sometimes even in the modern literature one finds the unfortunate *nonresidue* for a *nonsquare*. Legendre (1798) also proves Lemma 14.6 (ii) (Theorème 134, page 196).

[1] Let X be a polynomial that has no further multiple divisors. We have seen above that $x^p - x$ is the product of all irreducible polynomials of degree one. If ξ is the greatest common divisor of the polynomials X and $x^p - x$, then ξ will be the product of all divisors of X of degree one, and X/ξ will not have such factors any more. But if it is found that the polynomials X and $x^p - x$ are coprime, then X will have no divisor of degree one and hence the congruence $X \equiv 0$ will have no real [integer] roots. Moreover, since $x^{p^2} - x$ is the product of all irreducible polynomials of degrees two or one, the greatest common divisor ξ' of $x^{p^2} - x$ and X/ξ will contain all divisors of X of degree two. Continuing from here, one sees that X will be factored in this manner into factors ξ, ξ', ξ'' etc., which contain all [irreducible] divisors of degree one, two, three etc., respectively.

[2] We do not want to expand on this question, since a skilled calculator, well versed in these principles, will easily find special tricks when needed.

[3] One expands [...] $x^{(A-1)/2}$ in powers of x less than x^n, and sets this value equal to 1 and to -1. [...] Whenever the original equation has roots of both types [both squares and nonsquares], say p of the first type, satisfying $x^{(A-1)/2} = 1$ and q of the other type, satisfying $x^{(A-1)/2} = -1$, the separation of these types is achieved by the preceding method. [...] One can set $x = y \pm k$, k being arbitrary, and solve the equation in y by the same principles.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

The history of factorization algorithms involves many more people; see the references in the surveys of Kaltofen (1982, 1990, 1992) and von zur Gathen & Panario (2001), and in Shparlinski's (1992, 1999) treatises. Other early algorithms are by Prange (1959), Lloyd (1964), Lloyd & Remmers (1966), and Willett (1978). The survey by Slisenko (1981) mentions unpublished algorithms by Skopin and Faddeev, apparently found in the late 1960s.

If each $\deg g_i$ in Algorithm 14.3 is 0 or i, then the distinct-degree factorization is already the complete factorization of f. How often does this happen? We consider monic random polynomials in $\mathbb{F}_q[x]$ of degree n. Then, when n is fixed and $q \longrightarrow \infty$, the probability goes to $e^{-\gamma} \approx 56\%$ for large n, and for q fixed and $n \longrightarrow \infty$, this probability tends to a limit c_q, with $66.56\% \approx c_2 > c_q > e^{-\gamma}$ for all $q \geq 3$. This was shown by Flajolet, Gourdon & Panario (2001) (Theorem 4.1), who give further results on the distribution of factor degrees of random polynomials and the average case analysis of factoring algorithms. Similar and related results are in Knopfmacher & Knopfmacher (1993), Knopfmacher (1995), Knopfmacher & Warlimont (1995), Gourdon (1996), Gao & Panario (1997), Panario (1997), Panario, Gourdon & Flajolet (1998), Panario & Richmond (1998), and Panario & Viola (1998).

Gourdon (1996), Panario (1997), and Panario, Gourdon & Flajolet (1998) give results about the distributions of the degrees of the largest and the second largest irreducible factor of a random polynomial in $\mathbb{F}_q[x]$.

In general, an isomorphism between finite fields $\mathbb{F}_q \cong \mathbb{F}_p[x]/\langle f\rangle$ and $\mathbb{F}_q \cong \mathbb{F}_p[x]/\langle g\rangle$, where $f, g \in \mathbb{F}_p[x]$ are irreducible of degree n and $q = p^n$, can be obtained by mapping x mod f to a root of f in $\mathbb{F}_p[x]/\langle g\rangle$. Lenstra (1991) shows that such an isomorphism can even be constructed in deterministic polynomial time.

Instead of the norm $N(\alpha) = \alpha^{q^{d-1}+q^{d-2}+\cdots+1}$, one can also use the trace $T(\alpha) = \alpha^{q^{d-1}} + \alpha^{q^{d-2}} + \cdots + \alpha$ in equal-degree factorization; see McEliece (1969), Berlekamp (1970), Camion (1981, 1982, 1983), and von zur Gathen & Shoup (1992). Both functions have the crucial property that $N(\alpha), T(\alpha) \in \mathbb{F}_q$ for all $\alpha \in \mathbb{F}_{q^d}$. The trace also works in characteristic 2 (Exercise 14.16) where it is more tricky to apply the norm for factoring.

14.6. Gauß (1863b), article 368, basically describes the squarefree part algorithm 14.19, but does not deal with the difficulty when the characteristic divides all exponents (and the editor Dedekind repeats the incorrect statement). Lagrange (1769), §15, notes that $f/\gcd(f, f')$ has the same roots as f, but each with multiplicity one (over $\mathbb{C}$).

Algorithm 14.21 is from Yun (1976). For a random polynomial in $\mathbb{F}_q[x]$ of degree n, the expected degree of the squarefree part is asymptotically about $n - 1/q$ (Flajolet, Gourdon & Panario 2001, Theorem 2.1).

We have already noted that (4), saying that a polynomial with vanishing derivative over a finite field of characteristic $p > 0$ is a pth power, is not true for arbitrary fields of characteristic p. In fact, over sufficiently bizarre (but still "computable") fields it is undecidable—in the sense of Turing—whether a polynomial is squarefree or not (von zur Gathen (1984a), based on van der Waerden (1930a) and Fröhlich & Shepherdson (1955–56)). Van der Waerden's result is of particular interest because he has to assume explicitly that an undecidable problem—an "ignorabimus"—exists (this was proven by Turing later in 1937) and because Hilbert's (1930) article in the same volume of the *Mathematische Annalen* ends with Hilbert's credo: *In der Mathematik gibt es kein ignorabimus.*[4]

[4] There is no undecidable problem in mathematics.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

14.7. The iterated Frobenius algorithm is from von zur Gathen & Shoup (1992), where the estimate (7) is proven in their Theorem 3.2. This is also used in Kedlaya & Umans (2009).
14.8. The first pioneering random polynomial-time algorithms, based on linear algebra, are due to Berlekamp (1967, 1970). The matrix Q was already used by Petr (1937) who determined the characteristic polynomial of $Q-I$ and gave a distinct-degree factorization method using Q as representing the Frobenius automorphism. Schwarz (1939, 1940, 1956, 1960, 1961) and Butler (1954) used Q in various algorithms, for example to compute the number of factors of a given degree. Camion (1980) coined the term *Berlekamp algebra* for the kernel of β.

Berlekamp (1970) introduced the $(q-1)/2$ trick into modern polynomial factorization; Legendre already stated it in 1785.

A different linear algebra based method for factoring polynomials in $\mathbb{F}_q[x]$ was developed by Niederreiter (1993a, 1993b, 1994a), Göttfert (1994), and Niederreiter & Göttfert (1993, 1995); see Niederreiter (1994b) for an overview. The method turned out to be closely related to Berlekamp's algorithm. Gao & von zur Gathen (1994) showed how to combine it with Wiedemann's method. The special case where q is prime is discussed in Exercise 14.42. Gao (2003) extends Niederreiter's method to bivariate polynomials over a finite field.

Kaltofen & Shoup (1998) have found clever improvements to the factorization methods in this chapter that yield algorithms whose dependence on the degree of the polynomial to be factored is less than quadratic, namely $O(n^{1.815}(\log q)^{0.407})$ operations in $\mathbb{F}_q$. For practical purposes, they recommend a $O^{\sim}(n^{2.5}+n\log q)$ version of their method.

As Kaltofen and Shoup say in their "Note added in proof", their estimates can be improved slightly by combining them with the fast rectangular matrix multiplication algorithm of Huang & Pan (1998). This does not require any new algorithmic idea. As it is not in the literature, we briefly explain such an improvement, assuming familiarity with both papers. Theorem 10.2 of Huang & Pan gives an upper bound on $\omega(1,1,r)$, which is defined so that an $n\times n$ times $n\times n^r$ matrix product can be calculated with $O(n^{\omega(1,1,r)})$ arithmetic operations. This bound contains two parameters l and b. We set $l=7$ and $b=-0.00191r+0.03551$ in their bound and obtain a function $\varphi(r)$ with $\omega(1,1,r)\le\varphi(r)$. Then one verifies that $\varphi(r)\le 0.95732r+1.42261$ for $1.36437\le r\le 1.67555$. In Lemma 3 of Kaltofen & Shoup (1998), the dominating cost is a $t\times t$ times $t\times t^r$ matrix multiplication (more exactly, its transpose) with $t=n^{1/r}$ and $r=1/(1-\beta/2)$, for a parameter β. The cost for their algorithm is then $O^{\sim}(n^{\omega(1,1,r)/r}+n^{1+\beta+x})$, where $x=\log_n\log_2 q$, as in Figure 14.9. Using fast square matrix multiplication and the Coppersmith & Winograd exponent, we have $\omega(1,1,r)\le r-1+\omega(1,1,1)\le r+1.375477$. Equating the two exponents of n yields the value of β which minimizes the cost and provides the upper bound of $0.407x+1.815$ given in Kaltofen & Shoup (1998). Substituting the better linear bound on $\omega(1,1,r)$ for fast rectangular matrix multiplication from above and equating exponents, we find the upper bound $0.41565x+1.80636$, as in Figure 14.9. The required values of r all lie within the interval given above.

This estimate is not the best that one can get from the methods of Kaltofen & Shoup and Huang & Pan, but it is not clear to us how to obtain a simple explicit description of the running time that results from combining these methods in an optimal way. We do not claim that calculations as the above are of much value for practical purposes.

For large fields of small characteristic, say $\mathbb{F}_{2^k}$, Kaltofen & Shoup (1997) present even faster solutions by applying variants of the iterated Frobenius algorithm 14.26 over the

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

prime field. The natural cost measure now is to count word operations; as an example, they achieve $O(n(\log q)^{1.687})$ word operations when $k = \lceil n^{1.5} \rceil$.

14.9. The worst-case and average upper bounds on $\delta(n)$ are in Hardy & Wright (1985), §22.10. An exact formula for $I(n,q)$ and the approximation q^n/n are in Gauß (1863b), articles 344–347; the slightly sharper bound

$$\frac{q^n}{n} - \frac{q(q^{n/2}-1)}{(q-1)n} \leq I(n,q) \leq \frac{q^n - q}{n}$$

for $n \geq 2$ is in Lidl & Niederreiter (1997), Theorem 3.25 and Exercises 3.26 and 3.27. Algorithm 14.36 is due to Rabin (1980b), and Algorithm 14.40 to Ben-Or (1981). Lemma 14.41 was stated in Ben-Or (1981) and is proven in the solution to Exercise 7.32 of Bach & Shallit (1996). Panario & Richmond (1998) give a precise analysis of the implied constant. The expected minimal degree has a large variance, namely about cn for some constant $c \approx 0.5568$. Shepp & Lloyd (1966) proved a similar result about permutations, namely that the expected length of the shortest cycle of a random permutation on n letters is $O(\log n)$. Panario & Viola (1998) give an analysis of Rabin's algorithm. The estimate of the probability that a random polynomial over a finite field with no small factors is irreducible is from Gao & Panario (1997).

Galois (1830) proposed a probabilistic approach to finding irreducible polynomials over finite fields; see the quote at the beginning of this chapter. The asymptotically fastest method for computing irreducible polynomials is in Shoup (1994), using $O^{\sim}(n^2 + n \log q)$ operations in $\mathbb{F}_q$.

Further notes. The central open question in the theory of factoring polynomials over finite fields is: can this be done in deterministic polynomial time? We recall that the distinct-degree algorithm 14.3 is deterministic, but the equal-degree algorithm 14.10 is probabilistic. Thus we may assume $f \in \mathbb{F}_q[x]$ to be equal-degree. Berlekamp (1970) significantly simplified the problem: we may assume that q is a prime, and that f has only linear factors (see Exercise 14.40). Thus the question is the following:

> Given $f \in \mathbb{F}_p[x]$, of degree $n \leq p$, which is known to have n distinct roots in $\mathbb{F}_p$, and where p is prime, can we find these roots with a number of operations that is polynomial in n and $\log p$?

Several special cases have been solved: when $p-1$ has only small prime factors (so that $p-1$ is **smooth**, see Section 19.5) (Moenck 1977a, von zur Gathen 1987, Mignotte & Schnorr 1988), when $\Phi_k(p)$ is smooth for some cyclotomic polynomial $\Phi_k \in \mathbb{Z}[y]$ (Bach, von zur Gathen & Lenstra 2001), when f is cyclotomic when considered in $\mathbb{Q}[x]$ or, more generally, has commutative Galois group (Huang 1985, Rónyai 1989), or when n is small (Rónyai 1988). The most general result is Evdokimov's (1994) algorithm with an almost polynomial number of word operations $(n^{\log n} \log p)^{O(1)}$. All these results assume the Extended Riemann Hypothesis (ERH; see Notes 18.4). Irreducible polynomials can be computed in deterministic polynomial time under the ERH (Adleman & Lenstra 1986). Shoup (1990) and Lange & Winterhof (2000) present deterministic polynomial-time algorithms which factor almost all polynomials.

We stress that a solution of this interesting problem is unlikely to affect the practice of factoring, since there the probabilistic algorithms are just fine.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

Exercises.

14.1 (i) Let $\mathbb{F}_q$ be a finite field with q elements. Prove **Wilson's theorem** $\prod_{a\in\mathbb{F}_q^\times} a = -1$. Hint: Every $a \in \mathbb{F}_q^\times$ different from ± 1 has $a^{-1} \neq a$.

(ii) Prove a converse of Wilson's theorem: If n is an integer such that $(n-1)! \equiv -1 \bmod n$, then n is prime.

14.2 Suppose $p \geq 5$ is a prime, $f \in \mathbb{F}_p[x]$ has degree 4, and $\gcd(x^p - x, f) = \gcd(x^{p^2} - x, f) = 1$. What can you say about the factorization of f in $\mathbb{F}_p[x]$?

14.3 Trace Algorithm 14.3 on computing the distinct-degree decomposition of the squarefree polynomial

$$f = x^{17} + 2x^{15} + 4x^{13} + x^{12} + 2x^{11} + 2x^{10} + 3x^9 + 4x^8 + 4x^4 + 3x^3 + 2x^2 + 4x \in \mathbb{F}_5[x].$$

Tell from the output only how many irreducible factors of degree i the polynomial f has, for all i.

14.4 Let $q \in \mathbb{N}$ be a prime power.

(i) Use Theorem 14.2 to prove that if r is a prime number, then there are $(q^r - q)/r$ distinct monic irreducible polynomials of degree r in $\mathbb{F}_q[x]$. (Observe that, by Fermat's little theorem 4.9, $(q^r - q)/r$ is an integer.)

(ii) Now suppose that r is a prime power. Find a simple formula for the number of monic irreducible polynomials of degree r over $\mathbb{F}_q$.

14.5 Let $p \in \mathbb{N}$ be a prime and $f \in \mathbb{Z}[x]$ monic of degree n. Prove that the congruence $f(a) \equiv 0 \bmod p$ has n solutions $a \in \mathbb{Z}_p$ if and only $f \bmod p$ is a factor of $x^p - x$; that is, if and only if $x^p - x = fq + pr$, where q and r have integral coefficients, and where r is a polynomial of degree less than n.

14.6* Let q be a prime power and $f \in \mathbb{F}_q[x]$ squarefree of degree n.

(i) Prove that for $1 \leq a \leq b \leq n$, the polynomial

$$\gcd\left(\prod_{a\leq d<b} (x^{q^d} - x), f\right)$$

is the product of all monic irreducible factors of f whose degree divides some number in the interval $\{a, a+1, \ldots, b-1\}$.

(ii) Determine $\gcd\left(\prod_{a\leq d<b} (x^{q^b} - x^{q^{b-d}}), f\right)$.

(iii) Consider the following **blocking strategy** for distinct degree factorization. We partition the set $\{1, \ldots, n\}$ of possible degrees of irreducible factors of f into k intervals $I_1 = \{c_0 = 1, 2, \ldots, c_1 - 1\}$, $I_2 = \{c_1, c_1 + 1, \ldots, c_2 - 1\}$, …, $I_k = \{c_{k-1}, c_{k-1} + 2, \ldots, c_k - 1 = n\}$, with integers $1 = c_0 < c_1 < c_2 < \cdots < c_k = n + 1$. Describe an algorithm which, on input f, computes the polynomials $g_1, \ldots, g_k$ such that g_j is the product of all monic irreducible factors of f with degree in the interval I_j, for $1 \leq j \leq k$.

14.7 Show that -1 is a square in $\mathbb{F}_q^\times$ for an odd prime power q if and only if $q \equiv 1 \bmod 4$.

14.8 Let $\mathbb{F}_q$ be a finite field with q elements and $a, b \in \mathbb{F}_q^\times$ two nonsquares. Prove that ab is a square. Hint: Lemma 14.7.

14.9 If G is a group and $a, b \in G$, then b is a square root of a if $b^2 = a$.

(i) Prove that every element of a cyclic group G has at most two square roots.

(ii) Find a counterexample to (i) when G is not cyclic.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

14.10 We consider $\mathbb{F}_{41}$.

(i) Draw the "squaring graph" in $\mathbb{F}_{41}^\times$, the directed graph on vertices $1,\ldots,40$ with the edge (i,j) present if and only if $i^2 \equiv j \bmod p$, for $1 \le i,j \le 40$. Arrange your drawing so that the structure of the graph is easy to see.

(ii) Draw the "cubing graph" ($i^3 \equiv j \bmod p$).

(iii) Draw the "fifth power graph" ($i^5 \equiv j \bmod p$).

(iv) Can you see the qualitative differences in the three graphs above? Can you explain them?

(v) Let q be a prime larger than 1 000 000. What can you say about the "qth power graph" on $\mathbb{F}_{41}^\times$?

(vi) How many elements of $\mathbb{F}_{41}^\times$ are squares? How many are nonsquares? Same question for cubes and fifth powers.

14.11 This exercise generalizes Lemma 14.6. Let $\mathbb{F}_q$ be a finite field with q elements, and $k \in \mathbb{N}$.

(i) For $q = 13$ and $q = 17$, draw the graph of the cubing map $a \longmapsto a^3$ on $\mathbb{F}_q$, with the elements of $\mathbb{F}_q$ as vertices and an edge $a \longrightarrow b$ present if and only if $a^3 = b$.

(ii) Show that $\operatorname{ord}(a^k) = \operatorname{ord} a / \gcd(k, \operatorname{ord} a)$ for all $a \in \mathbb{F}_q^\times$.

(iii) Show that the kth power group homomorphism $\sigma_k\colon \mathbb{F}_q^\times \longrightarrow \mathbb{F}_q^\times$ is an automorphism if and only if $\gcd(k, q-1) = 1$. Hint: $\mathbb{F}_q^\times$ is cyclic (Exercise 8.16).

(iv) Conclude that $\ker \sigma_k = \ker \sigma_g$ and $\operatorname{im} \sigma_k = \operatorname{im} \sigma_g$, where $g = \gcd(k, q-1)$.

14.12 The squarefree polynomial

$$\begin{aligned} f = {} & x^{18} - 7x^{17} + 4x^{16} + 2x^{15} - x^{13} - 7x^{12} + 4x^{11} + 7x^{10} + 4x^9 \\ & -3x^8 - 3x^7 + 7x^6 - 7x^5 + 7x^4 + 7x^3 - 3x^2 + 5x + 5 \in \mathbb{F}_{17}[x] \end{aligned}$$

splits into 3 irreducible factors of degree 6.

(i) How would you check the above statement without factoring f, by computing at most three gcd's? (You need not actually compute the gcd's.)

(ii) Trace the equal-degree factorization algorithm 14.10 on computing these factors.

14.13 Write an efficient probabilistic algorithm that, given a prime p and an $a \in \mathbb{Z}_p^\times$, computes the square roots of a mod p—provided they exist. Apply your algorithm to $p = 2591$ and $a = 1005$.

14.15* Show that recursively applying the equal-degree splitting algorithm 14.8 to the smaller factor leads to an algorithm for finding *one* irreducible factor of the input polynomial with an expected running time of $O((d \log q + \log n)\mathsf{M}(n))$ operations in $\mathbb{F}_q$.

14.16* This exercise discusses variants of Algorithms 14.8 and 14.31 for finite fields of characteristic two. For $m \in \mathbb{N}$, we define the mth **trace polynomial** over $\mathbb{F}_2$ by $T_m = x^{2^{m-1}} + x^{2^{m-2}} + \cdots + x^4 + x^2 + x \in \mathbb{F}_2[x]$. Let $q = 2^k$ for some $k \in \mathbb{N}_{>0}$, $f \in \mathbb{F}_q[x]$ squarefree of degree n, with $r \ge 2$ irreducible factors $f_1,\ldots,f_r \in \mathbb{F}_q[x]$, $R = \mathbb{F}_q[x]/\langle f\rangle$, and $R_i = \mathbb{F}_q[x]/\langle f_i\rangle$ and $\chi_i\colon R \longrightarrow R_i$ with $\chi_i(a \bmod f) = a \bmod f_i$, for all i.

(i) Prove that $x^{2^m} + x = T_m \cdot (T_m + 1)$. Conclude that $T_m(\alpha) \in \mathbb{F}_2$ for any $\alpha \in \mathbb{F}_{2^m}$, and that both $T_m(\alpha) = 0$ and $T_m(\alpha) = 1$ occur with probability $1/2$ when α is chosen uniformly at random. Hint: The map induced by T_m is $\mathbb{F}_2$–linear.

(ii) Suppose that all irreducible factors of f have the same degree d. Show that $\chi_i(T_{kd}(\alpha)) \in \mathbb{F}_2$ for all $\alpha \in R$, and conclude that for a uniformly random $\alpha \in R$, we have $T_{kd}(\alpha) \in \mathbb{F}_2$ with probability $2^{1-r} \le 1/2$.

(iii) Modify the equal-degree splitting algorithm 14.8 so as to work for $q = 2^k$, by computing $b = T_{kd}(a)$ rem f in step 3. Prove that the modified algorithm fails with probability at most $1/2$, and that its running time is the same as that of the original algorithm.

(iv) Now let the degrees of the f_i be arbitrary again and $\mathcal{B} \subseteq R$ the Berlekamp subalgebra of R. Show that $\chi_i(T_k(\alpha)) \in \mathbb{F}_2$ for all $\alpha \in \mathcal{B}$, and conclude that for a uniformly random $\alpha \in \mathcal{B}$, we have $T_k(\alpha) \in \mathbb{F}_2$ with probability $2^{1-r} \le 1/2$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(v) Modify Berlekamp's algorithm 14.31 so as to work for $q = 2^k$, by computing $b = T_k(a) \bmod f$ in step 6. Prove that the modified algorithm fails with probability at most $1/2$, and that its running time is the same as that of the original algorithm.

14.17** The aim of this exercise is to reduce the expected cost estimate for equal degree factorization from $O((d\log q + \log n)\mathsf{M}(n)\log r)$ field operations, as shown in Theorem 14.9 and the discussion following it, to $O(d\log q \cdot \mathsf{M}(n) + \log(qn)\mathsf{M}(n)\log r)$.

Let q be an odd prime power and $f \in \mathbb{F}_q[x]$ squarefree of degree n with $r \geq 2$ irreducible factors $f_1, \ldots, f_r$ of degree $d = n/r$. We let $R, R_1, \ldots, R_r$ and the Chinese remainder isomorphism $\chi = \chi_1 \times \cdots \times \chi_r : R \longrightarrow R_1 \times \cdots \times R_r$ be as in Section 14.3. The **norm** on $R_i \cong \mathbb{F}_{q^d}$ is defined by $N(\alpha) = \alpha\alpha^q\alpha^{q^2}\cdots\alpha^{q^{d-1}} = \alpha^{(q^d-1)/(q-1)}$, and we use the same formula to define the norm on R.

(i) Let $\alpha \in R^\times$ be a uniform random element, $\beta = N(\alpha)$, and $1 \leq i \leq r$. Show that $\chi_i(\beta)$ is a root of $x^{q-1} - 1$, and conclude that $\chi_i(\beta)$ is a uniform random element in $\mathbb{F}_q^\times$. Hint: N is a homomorphism of multiplicative groups.

(ii) Provided that $q > r$, what is the probability that the $\chi_i(\beta)$ are distinct for $1 \leq i \leq r$? Prove that this probability is at least $1/2$ if $q - 1 \geq r^2$.

(iii) For $u \in \mathbb{F}_q$, let $\pi(u) = u^{(q-1)/2}$, so that $\pi(u) \in \{-1, 0, 1\}$, $\pi(u) = 0$ if and only if $u = 0$, and $\pi(u) = -1$ if and only if u is a nonsquare. Moreover, let $u, v \in \mathbb{F}_q$ be distinct. Prove that for a uniformly random $t \in \mathbb{F}_q$, we have $\pi(u+t) \neq \pi(v+t)$ with probability at least $1/2$. Hint: The map $t \longmapsto (u+t)/(v+t)$ if $t \neq -v$ and $-v \longmapsto 1$ is a bijection of $\mathbb{F}_q$.

(iv) Consider the following variant of Algorithm 14.8, due to Rabin (1980b).

ALGORITHM 14.54 Equal-degree splitting.

Input: A squarefree monic reducible polynomial $f \in \mathbb{F}_q[x]$ of degree n, where q is an odd prime power, a divisor $d < n$ of n, so that all irreducible factors of f have degree d, and $a \in \mathbb{F}_q[x]$ of degree less than n with $\chi_i(a \bmod f) \in \mathbb{F}_q$ for all i.

Output: A proper monic factor $g \in \mathbb{F}_q[x]$ of f, or "failure".

1. $g_1 \longleftarrow \gcd(a, f)$
   if $g_1 \neq 1$ and $g_1 \neq f$ **then return** g_1
2. choose $t \in \mathbb{F}_q$ at random
3. **call** the repeated squaring algorithm 4.8 in $R = \mathbb{F}_q[x]/\langle f \rangle$ to compute $b = (a+t)^{(q-1)/2} \bmod f$
4. $g_2 \longleftarrow \gcd(b - 1, f)$
   if $g_2 \neq 1$ and $g_2 \neq f$ **then return** g_2 **else return** "failure"

Use (iii) to prove that the failure probability of the algorithm is at most $1/2$ if $a \notin \mathbb{F}_q$.

(v) Use the algorithm from (iv) as a subroutine to create a recursive algorithm for equal-degree factorization, which has the same input specification as the above algorithm and outputs all irreducible factors of f. The value of a is passed to the recursive calls. Prove that the algorithm never halts if $\chi_i(a \bmod f) = \chi_j(a \bmod f)$ for some $i \neq j$, and that otherwise, if all $\chi_i(a \bmod f)$ are distinct elements of $\mathbb{F}_q$, the probability for its recursion depth to be more than $k = 1 + \lceil 2\log_2 r \rceil$ is at most $1/2$. Conclude that in the latter case, the number of operations in $\mathbb{F}_q$ is $O(\mathsf{M}(n)\log(qn)\log r)$.

(vi) Now we first compute $a = c^{(q^d-1)/(q-1)} \bmod f$ for a uniform random polynomial $c \in \mathbb{F}_q[x]$ of degree less than n, and then call the algorithm from (v) for that value of a and stop the recursion at depth k. We assume that $q - 1 \geq r^2$. Prove that with probability at least $1/4$, this method yields the r irreducible factors of f in time $O(d\mathsf{M}(n)\log q + \mathsf{M}(n)\log(qn)\log r)$.

14.18⟶ Use Algorithm 14.13 to factor the polynomial $x^6 + x^3 + x^2 + x + 1 \in \mathbb{F}_2[x]$ into irreducible factors. Show all your steps.

14.19 Let F be a field and $f \in F[x]$ with $f(0) \neq 0$. We recall $\mathrm{rev}(f) = f^* = x^{\deg f} f(1/x)$, the **reversal** (or reciprocal polynomial) of f (Section 9.1). We say that f is **self-reciprocal** if $f = f^*$.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(i) Show that * is multiplicative, so that $(fg)^* = f^*g^*$ for all $g \in F[x]$ with $g(0) \neq 0$.

(ii) Prove that $f(\alpha^{-1}) = 0 \iff f^*(\alpha) = 0$, for all $\alpha \in F$. Conclude that the set of zeroes of f is closed under inversion if f is self-reciprocal.

(iii) Show that every self-reciprocal polynomial f of odd degree satisfies $f(-1) = 0$.

(iv) Let $f \in F[x]$ with $f(0) \neq 0$ be self-reciprocal and $g \in F[x]$ an irreducible factor of f. Then also g^* is an irreducible factor of f.

(v) The squarefree polynomial $f = (x^{21}+1)/(x+1) \in \mathbb{F}_2[x]$ has—among others—the following irreducible factors: x^2+x+1, x^3+x+1, and $x^6+x^4+x^2+x+1$. What are the others?

14.20* Let $f \in \mathbb{F}_q[x]$ of degree n be given, and for $a \in \mathbb{F}_q$, let $B_a = \{b \in \mathbb{F}_q : f(b) = a\}$ be the set of preimages of a under the mapping $b \longmapsto f(b)$ induced by f.

(i) Given a, show how to compute $\prod_{b \in B_a}(y-b) \in \mathbb{F}_q[y]$ with $O(\mathsf{M}(n)\log(qn))$ operations in $\mathbb{F}_q$.

(ii) Given a, show how to compute probabilistically B_a with $O(\mathsf{M}(n)\log n \log(qn))$ operations in $\mathbb{F}_q$.

(iii) If the function corresponding to f is bijective (so that $\#B_a = 1$ for all $a \in \mathbb{F}_q$), then f is called a **permutation polynomial**. Use Exercise 14.11 to derive a criterion when $f = x^n$ is a permutation polynomial.

(iv) If f is not a permutation polynomial, then in fact

$$\#\{a : B_a \neq \emptyset\} = \#\operatorname{im} f \leq q\left(1 - \frac{1}{n}\right)$$

(Wan 1993; a weaker result is in von zur Gathen 1991b). Use this fact to derive a probabilistic (Monte Carlo) test for permutation polynomials, taking $O(n\mathsf{M}(n)\log(qn))$ operations in $\mathbb{F}_q$.

14.21 Let $f \in \mathbb{Z}[x]$ be of degree n and max-norm $\|f\|_\infty = A$, and $f = (ux+v)g$, with nonzero $u, v \in \mathbb{Z}$ and $g = \sum_{0 \leq i < n} g_i x^i \in \mathbb{Z}[x]$.

(i) Prove that $|g_i| \leq (i+1)A/|v|$ for $0 \leq i < n-1$ if $|u| = |v|$, and conclude that then $\|g\|_\infty \leq nA$.

(ii) Now assume that $\alpha = |u/v| < 1$. Show that $|g_i| \leq A(1-\alpha^{i+1})/(1-\alpha)|v|$ for $0 \leq i < n-1$, and conclude that $\|g\|_\infty \leq A$. Prove that the latter also holds if $|u/v| > 1$.

14.22 (i) Use the Leibniz rule to prove (2).

(ii) Conclude that $f/\gcd(f,f') = \prod_{e_i f_i' \neq 0} f_i$.

14.23 Prove or disprove:

(i) The polynomial $x^{1000}+2 \in \mathbb{F}_5[x]$ is squarefree.

(ii) Let F be a field and $f, g \in F[x]$. Then the squarefree part of fg is the product of the squarefree parts of f and of g.

14.24 (Yun 1977b) Over a field F of characteristic zero, Algorithm 14.19 reduces the problem of computing the squarefree part of a polynomial to a gcd computation.

(i) Show that conversely computing a gcd of two squarefree polynomials $f, g \in F[x]$ can be reduced to computing the squarefree part of a certain polynomial.

(ii) Let $f, g \in F[x]$ be monic nonconstant, with squarefree decompositions $f = \prod_{1 \leq i \leq m} f_i^i$ and $g = \prod_{1 \leq i \leq k} g_i^i$. Show that $\gcd(f,g) = \prod_{1 \leq i \leq \min\{m,k\}} \gcd(f_i \cdots f_m, g_i \cdots g_k)$, and conclude from (i) that computing gcd's can be reduced to computing squarefree decompositions.

14.25 Test the following polynomials for multiple factors in $\mathbb{Q}[x]$.

(i) $x^3 - 3x^2 + 4$, (ii) $x^3 - 2x^2 - x + 2$.

14.26 Let F be a field of characteristic zero, $f \in F[x]$ monic nonconstant of degree n, $f = g_1 g_2^2 \cdots g_m^m$ its squarefree decomposition, $v = g_1 \cdots g_m$, $u = f/v$, and $w = f'/u$.

(i) Show that $\gcd(f,f') = u$ and $w = \sum_{1 \leq i \leq m} i g_i' \dfrac{v}{g_i}$. Hint: Exercise 14.22.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(ii) Prove that $g_i = \gcd(v, w - iv')$ for $1 \le i \le m$.

(iii) State the squarefree factorization algorithm resulting from (ii). Show that its running time in field operations is $O(n\mathsf{M}(n)\log n)$, and give an example where this upper bound is essentially achieved.

14.27* Let $\mathbb{F}_q$ be a finite field of characteristic p, and $f = f_1^{e_1} \cdots f_r^{e_r} \in \mathbb{F}_q[x]$ of degree n, with distinct monic irreducible polynomials $f_1, \ldots, f_r \in \mathbb{F}_q[x]$ and positive $e_1, \ldots, e_r \in \mathbb{N}$.

(i) Show that Algorithm 14.19 returns $v = \prod_{p \nmid e_i} f_i$.

(ii) Prove that $u/\gcd(u, v^n) = \prod_{p \mid e_i} f_i^{e_i}$, and show how to compute it at a cost of $O(\mathsf{M}(n)\log n)$ operations in $\mathbb{F}_q$.

(iii) Derive an algorithm for computing the squarefree part over $\mathbb{F}_q$ by computing a pth root of $u/\gcd(u, v^n)$ and proceeding recursively. Show that this algorithm uses $O(\mathsf{M}(n)\log n + n\log(q/p))$ operations in $\mathbb{F}_q$.

(iv) Let $f = ab^2c^2d^6e^8 \in \mathbb{F}_2[x]$, with irreducible and pairwise coprime polynomials $a, b, c, d, e \in \mathbb{F}_2[x]$. Use your algorithm from (iii) to compute its squarefree part.

14.28 Prove that the squarefree decomposition of a monic polynomial is unique.

14.29 Compute the squarefree decomposition of the following polynomials in $\mathbb{Q}[x]$ and in $\mathbb{F}_3[x]$.

(i) $x^6 - x^5 - 4x^4 + 2x^3 + 5x^2 - x - 2$,

(ii) $x^6 - 3x^5 + 6x^3 - 3x^2 - 3x + 2$,

(iii) $x^5 - 2x^4 - 2x^3 + 4x^2 + x - 2$,

(iv) $x^6 - 2x^5 - 4x^4 + 6x^3 + 7x^2 - 4x - 4$,

(v) $x^6 - 6x^5 + 12x^4 - 6x^3 - 9x^2 + 12x - 4$.

14.30* Let $\mathbb{F}_q$ be a finite field of characteristic p, and $f \in \mathbb{F}_q[x]$ monic and nonconstant with squarefree decomposition $(g_1, \ldots, g_m)$.

(i) Show that Yun's algorithm 14.21 returns the correct result if $m < p$.

(ii) Let $m \ge p$ and show that Algorithm 14.21 when applied to f computes $h_i = \prod_{j \equiv i \bmod p} g_j$ for $1 \le i < p$ and $h_i = 1$ for $i \ge p$.

(iii) Modify Algorithm 14.21 so as to work correctly also for $m \ge p$. (Hint: $fh_1^{-1}h_2^{-2}\cdots h_{p-1}^{-p+1}$ is a pth power.) Show that the modified algorithm takes $O(\mathsf{M}(n)\log n + n\log(q/p))$ operations in $\mathbb{F}_q$. Trace the algorithm on the example polynomial from Exercise 14.27 (iv).

14.31 Let F be a field, and let $f, g_1, \ldots, g_m \in F[x]$ be monic nonconstant polynomials. Recall that $(g_1, \ldots, g_m)$ is the squarefree decomposition of f if $f = g_1 g_2^2 \cdots g_m^m$, each g_i is squarefree, the g_i are pairwise coprime, and $g_m \ne 1$.

(i) Prove that there is a unique decomposition $f = h_1 \cdots h_m$ such that each h_i is monic, nonconstant, and squarefree, $h_i \mid h_{i-1}$ for $2 \le i \le m$, and $h_m \ne 1$.

(ii) Give both decompositions for $f = x^4(x+1)^3$.

(iii) Express the h_i in terms of the g_i and vice versa, and show that both conversions can be computed in time $O(\mathsf{M}(n))$ if $n = \deg f$.

14.32 (Carlitz 1932) You are to show that for a prime power q and a positive integer $n \ge 2$, the probability for a random polynomial in $\mathbb{F}_q[x]$ of degree n to be squarefree is $1 - 1/q$. Let s_n denote the number of monic squarefree polynomials of degree n in $\mathbb{F}_q[x]$. Then $s_0 = 1$ and $s_1 = q$.

(i) Prove the recursive formula $\sum_{0 \le 2k \le n} q^k s_{n-2k} = q^n$. Hint: Every monic polynomial $f \in \mathbb{F}_q[x]$ can be uniquely written as $f = g^2 h$ with a squarefree monic polynomial h.

(ii) Conclude that $s_n = q^n - q^{n-1}$ if $n \ge 2$ by subtracting a suitable multiple of the above formula for $n-2$ from the formula itself.

14.33 Let F be a field of positive characteristic p and $a \in F$ such that a has no pth root in F. Prove that $x^p - a \in F[x]$ is irreducible. Hint: $f = (x - a^{1/p})^p$ over its splitting field $F(a^{1/p})$. Look at the coefficient of x^{n-1} of a hypothetical factor $g \in F[x]$ of degree n of f.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

14.34 Let $\mathbb{F}_q$ be a finite field with q elements, $f \in \mathbb{F}_q[x]$ nonconstant, and $\xi = x^q \bmod f \in R = \mathbb{F}_q[x]/\langle f\rangle$. Prove or disprove that $\alpha^q = \check{\xi}(\alpha)$ for all $\alpha \in R$.

14.35* Find "small" constants $c_1, c_2 \in \mathbb{Q}$ such that the running time of the iterated Frobenius algorithm 14.26 is at most $(c_1 n/d + c_2)\mathsf{M}(d)\log_2 d + O(\mathsf{M}(d) + n\log d)$ additions and multiplications in R when n and d are powers of 2. Hint: Exercise 10.2.

14.36* Let q be a prime power, $f \in \mathbb{F}_q[x]$ of degree n, and $R = \mathbb{F}_q[x]/\langle f\rangle$.

(i) Consider the following algorithm for computing the norm $N_d(\alpha) = \alpha\alpha^q \cdots \alpha^{q^{d-1}}$ for $\alpha \in R$ and a power of two $d < n$.

ALGORITHM 14.55 Norm computation.

Input: $f \in \mathbb{F}_q[x]$ of degree n, a power of two $d \in \mathbb{N}$ with $d \le n$, $\xi^q \in R = \mathbb{F}_q[x]/\langle f\rangle$, where $\xi = x \bmod f \in R$, and $\alpha \in R$.
Output: $N_d(\alpha) \in R$.

1. $\gamma_0 \longleftarrow \xi^q, \quad \delta_0 \longleftarrow \xi, \quad l \longleftarrow \log_2 d$
2. **for** $i = 1, \ldots, l$ **do**
   call the modular composition algorithm 12.3 to compute $\gamma_i = \check{\gamma}_{i-1}(\gamma_{i-1})$ and $\check{\delta}_{i-1}(\gamma_{i-1})$
   $\delta_i \longleftarrow \delta_{i-1} \cdot \check{\delta}_{i-1}(\gamma_{i-1})$
3. **return** δ_l

Prove that the algorithm works correctly and takes $O((n^{(\omega+1)/2} + n^{1/2}\mathsf{M}(n))\log d)$ operations in $\mathbb{F}_q$. Compare this to the time for computing the norm by employing the iterated Frobenius algorithm 14.26.

(ii) Modify the algorithm so as to also work when d is not necessarily a power of two.

(iii) Design a similar algorithm for computing the trace $T_d(\alpha) = \alpha + \alpha^q + \cdots + \alpha^{q^{d-1}}$ of $\alpha \in R$.

14.37 Let q be a prime power, $f = f_1 \cdots f_r \in \mathbb{F}_q[x]$ squarefree, with monic irreducible and pairwise coprime $f_1, \ldots, f_r \in \mathbb{F}_q[x]$, and $\mathcal{B} \subseteq \mathbb{F}_q[x]/\langle f\rangle$ the Berlekamp algebra of f. Prove that the "Lagrange interpolants" $l_1, \ldots, l_r \in \mathbb{F}_q[x]$ of degree less than $\deg f$ and with $l_i \equiv 0 \bmod f_j$ if $j \ne i$ and $l_i \equiv 1 \bmod f_i$ are a basis of $\mathcal{B}$.

14.38* Let $f = f_1 \cdots f_r \in \mathbb{F}_2[x]$ be squarefree of degree n, with $f_1, \ldots, f_r \in \mathbb{F}_2[x]$ monic irreducible and pairwise coprime, $\mathcal{B} \subseteq \mathbb{F}_2[x]/\langle f\rangle$ its Berlekamp algebra, and $b_1 \bmod f, \ldots, b_r \bmod f$ a basis of $\mathcal{B}$, with all $b_i \in \mathbb{F}_2[x]$ of degree less than n.

(i) Show that for $1 \le i \le r$ with at most one exception, there exist indices j, k such that $f_i \mid b_j$ and $f_i \nmid b_k$.

(ii) Let $f = g_1 \cdots g_s$ be a partial factorization of f, with all g_j monic nonconstant and pairwise coprime, and $1 \le i \le r$. Use Exercise 11.4 to show that

$$\gcd(b_i, g_1), \frac{g_1}{\gcd(b_i, g_1)}, \ldots, \gcd(b_i, g_s), \frac{g_s}{\gcd(b_i, g_s)}$$

can be computed using $O(\mathsf{M}(n)\log n)$ operations in $\mathbb{F}_2$ (we call this a refinement with b_i).

(iii) Show that by successively refining partial factorizations of f with $b_1, \ldots, b_r$, starting with the trivial factorization $f = f$, we obtain all irreducible factors of f in time $O(r \cdot \mathsf{M}(n)\log n)$.

14.39* Let $p \in \mathbb{N}$ be prime and $q = p^k$ for some positive $k \in \mathbb{N}$, $f \in \mathbb{F}_q[x]$ monic squarefree of degree n, and $R = \mathbb{F}_q[x]/\langle f\rangle$. We may replace the Frobenius endomorphism $\alpha \longmapsto \alpha^q$ of R over $\mathbb{F}_q$ in Berlekamp's algorithm 14.31 by the **absolute Frobenius endomorphism** $\alpha \longmapsto \alpha^p$ of R over the prime field $\mathbb{F}_p$. Analyze this variant and compare its expected running time to that of the original algorithm.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

14.40* It is clear that finding roots of polynomials is a special case of factoring polynomials. This exercise shows conversely how factoring over a finite field can be reduced to root finding over the prime field. Let $q = p^k$ be a prime power for some positive $k \in \mathbb{N}$, $f \in \mathbb{F}_q[x]$ monic squarefree of degree n, $R = \mathbb{F}_q[x]/\langle f\rangle$, and $\mathcal{B} = \{a \bmod f \in R : a^p \equiv a \bmod f\} \subseteq R$ the absolute Berlekamp subalgebra (see Exercise 14.39).

(i) Let $b \in \mathbb{F}_q[x]$ such that $b \bmod f \in \mathcal{B}$. Prove that $f = \prod_{a \in \mathbb{F}_p} \gcd(f, b-a)$.

(ii) Let y be a new indeterminate and $r = \operatorname{res}_x(f, b-y) \in \mathbb{F}_q[y]$. Show that r has some roots in $\mathbb{F}_p$, and that any root of r in $\mathbb{F}_p$ leads to a nontrivial factor of f if $b \notin \mathbb{F}_p$.

(iii) Give a deterministic polynomial-time reduction from factoring in $\mathbb{F}_q[x]$ to root finding in $\mathbb{F}_p[x]$.

14.41* Let q be a prime power, $f \in \mathbb{F}_q[x]$ monic squarefree of degree n, and $R = \mathbb{F}_q[x]/\langle f\rangle$, as usual.

(i) Show that if f splits into r irreducible factors of degrees $d_1, \ldots, d_r$, then $\operatorname{lcm}\{x^{d_i} - 1 : 1 \le i \le r\}$ is the minimal polynomial of the matrix Q representing the Frobenius endomorphism $\alpha \longmapsto \alpha^q$ on R. Hint: Start with $r = 1$. Conclude that f is irreducible if and only if the minimal polynomial of Q is $x^n - 1$.

(ii) Use (i) and Exercise 12.15 to design a Monte Carlo test whether f is irreducible. Your test should take $O(n \cdot \mathsf{M}(n) \log q)$ operations in $\mathbb{F}_q$ if q is "large enough".

14.42** This exercise discusses the easiest case of another factoring method based on linear algebra, due to Niederreiter (see Notes 14.8). Let $p \in \mathbb{N}$ be prime.

(i) Prove that for all rational functions $h \in \mathbb{F}_p(x)$, the $(p-1)$st derivative $h^{(p-1)}$ is a pth power.

(ii) Show that for any nonzero polynomial $f \in \mathbb{F}_p[x]$, the rational function $h = f'/f \in \mathbb{F}_p(x)$ is a solution of the differential equation

$$h^{(p-1)} + h^p = 0. \tag{14}$$

Hint: Prove this first when f is squarefree, using Exercise 9.27 over the splitting field of f, and Wilson's theorem (Exercise 14.1). For the general case, employ the squarefree decomposition of f and Exercise 9.27.

(iii) Prove that if $h = g/f \in \mathbb{F}_p(x)$ satisfies (14), with nonzero coprime $f, g \in \mathbb{F}_p[x]$ and f monic, then $\deg g < \deg f$ and f is squarefree.

(iv) Let f, g be as in (iii) and $\lambda_1, \ldots \lambda_n \in E$ the (distinct) roots of f in a splitting field E of f over $\mathbb{F}_p$. By partial fraction decomposition, there exist $d_1, \ldots, d_n \in E$ such that

$$\frac{g}{f} = \sum_{1 \le i \le n} \frac{d_i}{x - \lambda_i}.$$

Show that $y = d_i/(x - \lambda_i)$ solves (14) for $1 \le i \le n$. (Hint: Uniqueness of partial fraction decomposition). Prove that $d_i = d_k \in \mathbb{F}_p$ if λ_i and λ_k are roots of the same irreducible factor in $\mathbb{F}_p[x]$ of f, and conclude that

$$\frac{g}{f} = \sum_{1 \le j \le r} c_j \frac{f_j'}{f_j}$$

for some $c_1, \ldots, c_r \in \mathbb{F}_p$, where $f_1, \ldots, f_r$ are the distinct monic irreducible factors of f.

(v) Let $f \in \mathbb{F}_p[x]$ be monic of degree n with the factorization $f = f_1^{e_1} \cdots f_r^{e_r}$ into irreducible factors, and

$$\mathcal{N} = \{g \in \mathbb{F}_p[x] : \deg g < n \text{ and } h = \frac{g}{f} \text{ satisfies (14)}\}.$$

Prove that $f_1' f/f_1, \ldots, f_r' f/f_r$ is a basis of $\mathcal{N}$ as a vector space over $\mathbb{F}_p$.

(vi) Now let f be squarefree and $\mathcal{B} \subseteq \mathbb{F}_p[x]/\langle f\rangle$ the Berlekamp algebra of f. Prove that the map $\varphi : \mathcal{N} \longrightarrow \mathcal{B}$ with $\varphi(g) = g \cdot (f')^{-1} \bmod f$ is a vector space isomorphism. Hint: Consider $\varphi(g) \bmod f_j$ for all j.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(vii) Assume that $p > 2$. Let f as in (vi), $g = \sum_{1 \le j \le r} c_j f_j' f / f_j \in \mathcal{N}$ with all $c_i \in \mathbb{F}_p$, and $S \subseteq \mathbb{F}_p^\times$ the set of squares. Show that

$$\gcd(g^{(p-1)/2} - (f')^{(p-1)/2}, f) = \prod_{c_j \in S} f_j,$$

and conclude that this gcd is nontrivial with probability at least $1/2$ if $c_1, \ldots, c_r$ are chosen uniformly at random in $\mathbb{F}_p$ and $\gcd(f, g) = 1$.

14.43** This exercise turns the theory from Exercise 14.42 into an algorithm for $p = 2$. Let $f = \sum_{0 \le i \le n} f_i x^i \in \mathbb{F}_2[x]$ be monic squarefree of degree n.

(i) Prove that $\mathcal{N} = \{g \in \mathbb{F}_2[x] : \deg g < n \text{ and } (fg)' = g^2\}$.

(ii) Let $N \in \mathbb{F}_2^{n \times n}$ be the matrix of the linear operator $g \longmapsto \left((fg)'\right)^{1/2}$ on the vector space of all polynomials in $\mathbb{F}_2[x]$ of degree less than n with respect to the polynomial basis $x^{n-1}, x^{n-2}, \ldots, x, 1$, so that

$$(f \sum_{0 \le i < n} g_i x^i)' = \left(\sum_{0 \le i < n} h_i x^i \right)^2 \iff N \cdot \begin{pmatrix} g_{n-1} \\ \vdots \\ g_0 \end{pmatrix} = \begin{pmatrix} h_{n-1} \\ \vdots \\ h_0 \end{pmatrix}.$$

Prove that

$$N = \begin{pmatrix} f_n & 0 & 0 & 0 & 0 & 0 & \cdots \\ f_{n-2} & f_{n-1} & f_n & 0 & 0 & 0 & \cdots \\ f_{n-4} & f_{n-3} & f_{n-2} & f_{n-1} & f_n & 0 & \cdots \\ \vdots & & & & & & \vdots \end{pmatrix}.$$

(iii) Design an algorithm for factoring f by determining a basis of $\mathcal{N} - I$, where I is the $n \times n$ identity matrix, and using an analog of Exercise 14.38. Prove that it takes $O(n^\omega)$ operations in $\mathbb{F}_2$, like Berlekamp's algorithm.

14.44* Let q be a prime power, $t \in \mathbb{N}$ a prime divisor of $q - 1$, and $a \in \mathbb{F}_q^\times$.

(i) Show that the polynomial $x^t - a \in \mathbb{F}_q[x]$ splits into linear factors if a is a tth power Hint: Use Lemma 8.8.

(ii) Show that $x^t - a$ is irreducible if a is not a tth power Hint: Use (i) for the splitting field of $x^t - a$ and consider the constant coefficient of a hypothetical factor $f \in \mathbb{F}_q[x]$ of $x^t - a$.

(iii) Derive a formula for the probability that a random binomial $x^t - a$ (that is, for random $a \in \mathbb{F}_q^\times$) is irreducible, and compare it to the probability that a random polynomial of degree t in $\mathbb{F}_q[x]$ is irreducible.

14.45 Prove Lemma 14.47.

14.46 This exercise discusses a useful tool from number theory: **Möbius inversion**. Let the Möbius function μ be defined as in (11).

(i) Prove that μ is multiplicative, so that $\mu(mn) = \mu(m)\mu(n)$ whenever $m, n \in \mathbb{N}_{>0}$ are coprime.

(ii) Show that $\sum_{d|n} \mu(d) = 0$ if $n > 1$, where the sum is over all positive divisors of n.

(iii) Let R be an arbitrary ring (commutative, with 1) and $f, g : \mathbb{N}_{>0} \longrightarrow R$ be two functions such that

$$f(n) = \sum_{d|n} g(d) \text{ for } n \in \mathbb{N}_{>0}.$$

Prove that

$$g(n) = \sum_{d|n} \mu\left(\frac{n}{d}\right) f(d) = \sum_{d|n} \mu(d) f\left(\frac{n}{d}\right) \text{ for } n \in \mathbb{N}_{>0}.$$

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

(iv) Now assume that

$$f(n) = \prod_{d|n} g(d) \text{ for } n \in \mathbb{N}_{>0}.$$

Give a similar formula for g in terms of μ and f as in (iii).

(v) Let $d(n)$ denote the number of positive divisors of $n \in \mathbb{N}_{>0}$. Prove that $\sum_{e|n} \mu(n/e)d(e) = 1$ for all positive integers n.

14.47* (Prange 1959) Let $q \in \mathbb{N}$ be a prime power coprime to $n \in \mathbb{N}$. We use the equivalence relation $\sim$ as in (13). Let $S_1, \ldots, S_r$ be the distinct equivalence classes of $\mathbb{Z}_n$ with respect to $\sim$, and $b_i = \sum_{j \in S_i} x^j \in \mathbb{F}_q[x]$ for $1 \leq i \leq r$.

(i) Show that a basis of the Berlekamp algebra $\mathcal{B} \subseteq \mathbb{F}_q[x]/\langle x^n - 1\rangle$ belonging to the polynomial $x^n - 1$ is given by $b_1 \bmod x^n - 1, \ldots, b_r \bmod x^n - 1$.

(ii) Give an algorithm which computes all irreducible factors of the polynomial $x^n - 1$ in $\mathbb{F}_q[x]$ using $O(\log q \log n + n\mathsf{M}(\log n))$ word operations and an expected number of $O(\mathsf{M}(n)\log(qn)\log r)$ field operations in $\mathbb{F}_q$, in total $O^{\sim}(n\log^2 q)$ word operations. Hint: Use the method of Exercise 11.4 to split factors already found and perform a similar analysis as in the proof of Theorem 14.11.

Research problems.

14.48 Find a deterministic polynomial-time algorithm for computing a root of a squarefree polynomial $f \in \mathbb{F}_p[x]$ which divides $x^p - x$, where p is a prime number. (Exercise 14.40 implies that then the general problem of factoring polynomials can be solved in deterministic polynomial time.)

14.49 Allowing probabilistic algorithms using only operations in $\mathbb{F}_q$, show that $\Omega(n \log q)$ operations are required to factor a polynomial of degree n in $\mathbb{F}_q[x]$. This corresponds to the diagonal $y = x + 1$ in Figure 14.9.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press

The operation of factoring [polynomials]
must be performed by inspection.

Charles Davies (1867)

Tous les effets de la nature ne sont que les résultats
mathématiques d'un petit nombre de lois immuables.[1]

Pierre Simon Laplace (1812)

[1] All the effects of nature are only mathematical results of a small number of immutable laws.

https://doi.org/10.1017/CBO9781139856065.018 Published online by Cambridge University Press